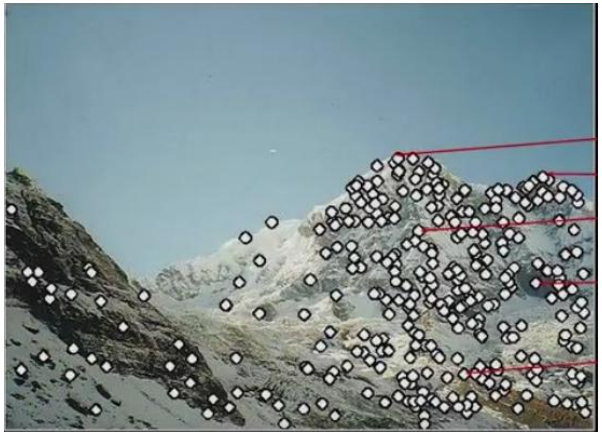
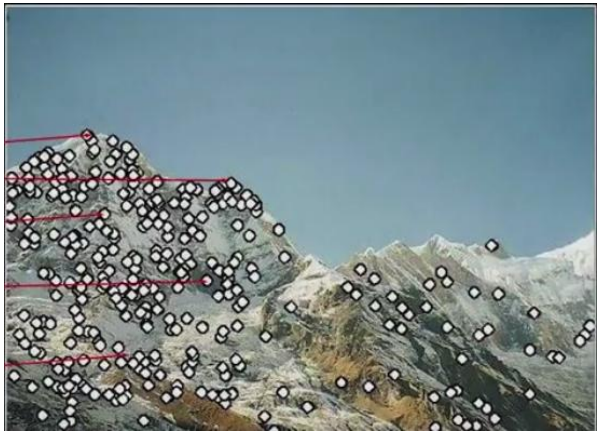
Keypoint e descrittori locali

Annalisa Franco
annalisa.franco@unibo.it

Cosa serve per allineare immagini?



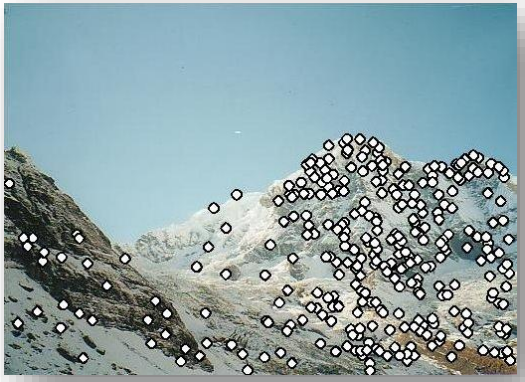




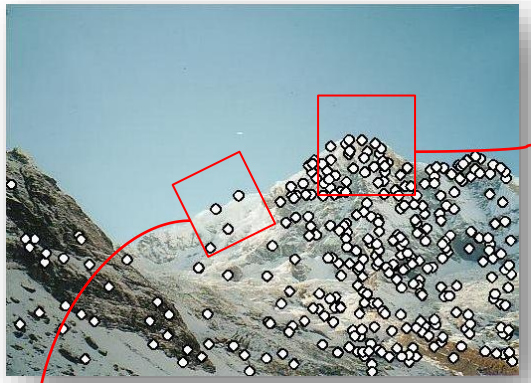
Soluzione con feature locali…



Localizzazione dei punti di interesse



Calcolo di un descrittore per ciascun punto



$\mathbf{x}_1 = [x_1^{(1)}, \ldots, x_d^{(1)}]$

$\mathbf{x}_2 = [x_1^{(2)}, \ldots, x_d^{(2)}]$

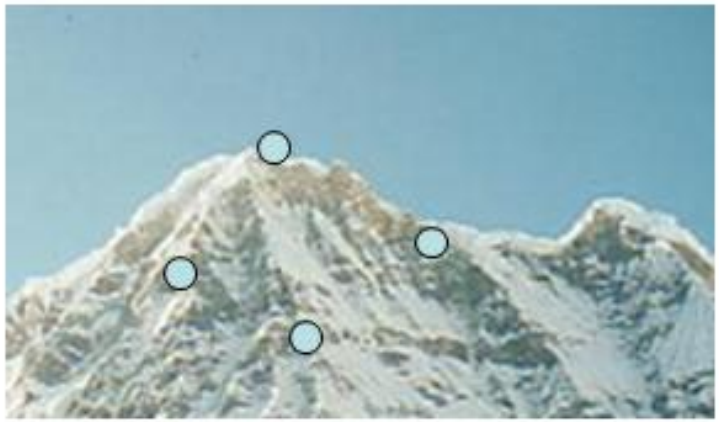
Matching (corrispondenza tra descrittori)



Requisiti delle feature invariant (1)

Delle buone feature locali dovrebbero possedere le seguenti caratteristiche:

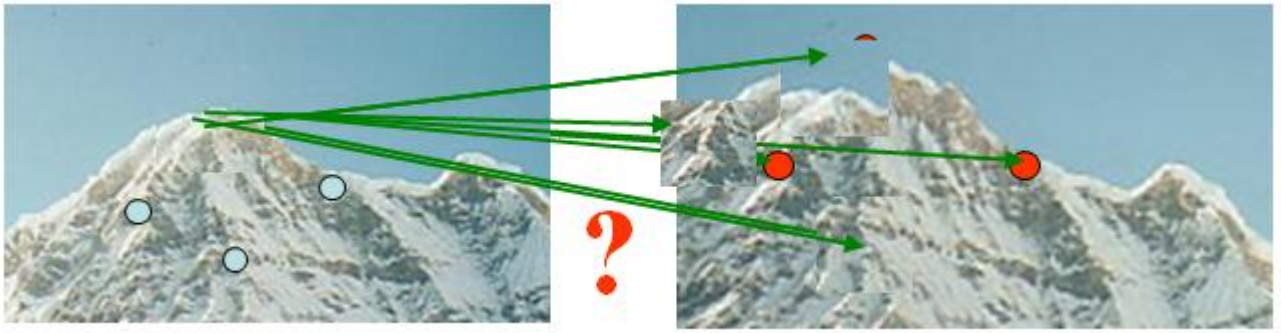
- Ripetibilità: le stesse feature possono essere individuate in immagini diverse, nonostante possibili trasformazioni geometriche e fotometriche;





Requisiti delle feature invariant (2)

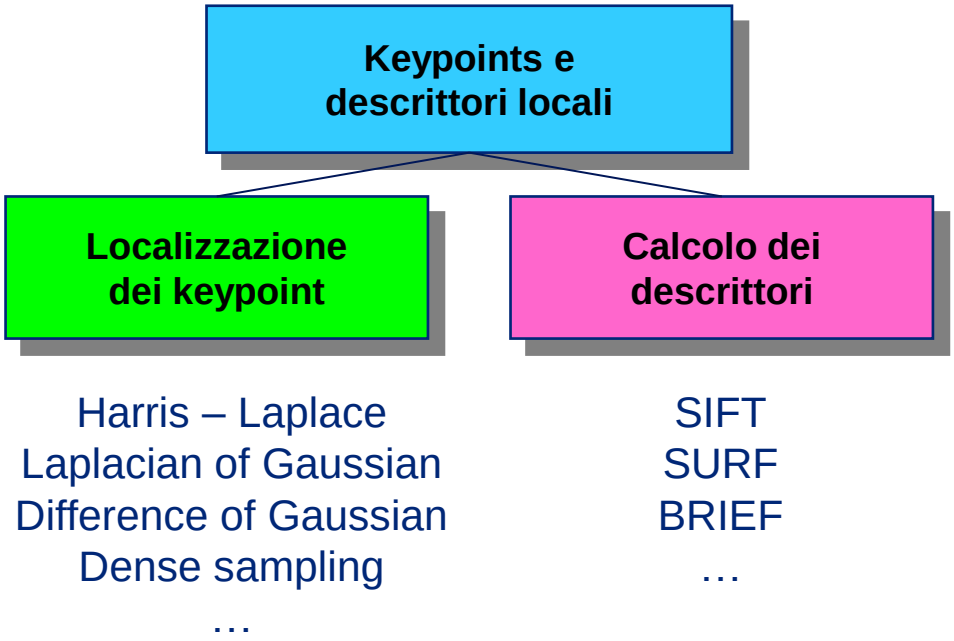
- Rilevanza: ciascuna feature deve essere significativa e rappresentativa della porzione di immagine che la circonda e permettere di effettuare un matching significativo;



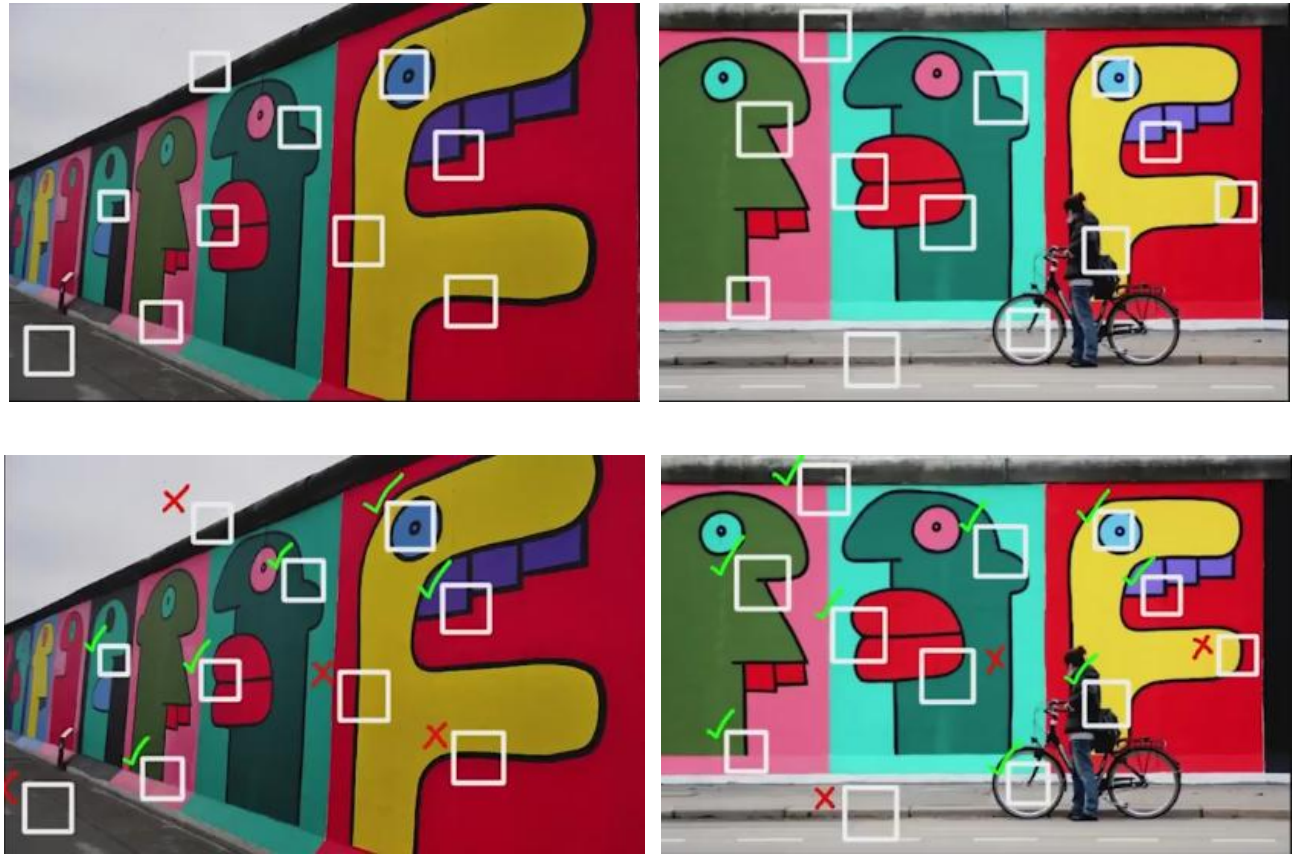
- Compattezza ed efficienza: il numero di feature da utilizzare per rappresentare un'immagine deve essere molto inferiore rispetto al numero di pixel dell'immagine stessa;
- Località: ogni feature deve occupare una porzione relativamente piccola dell'immagine; ciò la rende maggiormente robusta dispetto a occlusioni o presenza di rumore.

Keypoints e descrittori locali

- L'idea alla base di questa categoria di metodi è quella di individuare alcuni punti notevoli (keypoints) all'interno dell'immagine e di associare a ciascuno un descrittore locale che riassuma le informazioni salienti presenti in un intorno del punto. Per rendere l'approccio più robusto si una solitamente un'analisi multi-scala che permette di gestire meglio la variabilità che può caratterizzare le immagini.



Good features



Localizzazione di keypoint

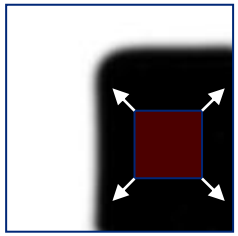


Quali punti scegliere?

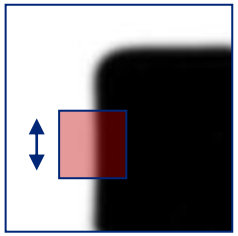
Localizzazione di keypoint

I punti di interesse dovrebbero essere facilmente riconoscibili, analizzando una piccola porzione di immagine nel loro intorno.

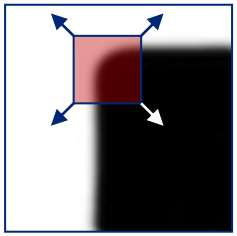
Uno spostamento anche piccolo della finestra di osservazioni in qualsiasi direzione dovrebbe portare a un cambiamento notevole di intensità.



Nelle regioni flat non si rilevano cambiamenti di intensità in nessuna direzione



In corrispondenza di edge si rilevano cambiamenti di intensità nella direzione ortogonale dell'edge



Nei corner si misurano cambiamenti significativi di intensità in tutte le direzioni

Harris corner detector [Harris88]

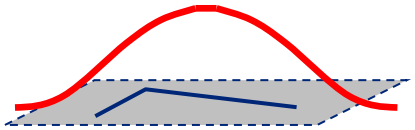
- L'idea alla base del metodo è individuare i corner come i punti in cui si incontrano due edge, caratterizzati da un gradiente elevato in entrambe le direzioni (orizzontale e verticale).
- Per la localizzazione dei corner si può procedere analizzando la distribuzione delle componenti del gradiente in sottofinestre dell'immagine per differenziare aree flat, edge e corner. In particolare il cambiamento di intensità all'interno di una finestra in seguito a una traslazione di (u, v) è dato da:

$$E(u,v) = \sum_{x,y} w(x,y)[I(x+u, y+v) - I(x,y)]^2$$

Window function

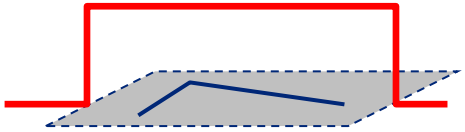
Shifted intensity

Intensity



Gaussiana

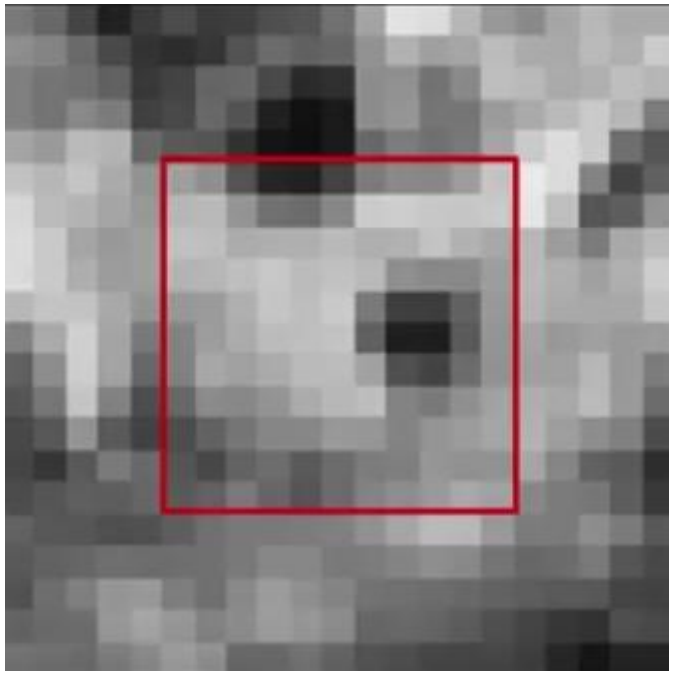
Window function



1 dentro la finestra
0 fuori

Harris corner detector

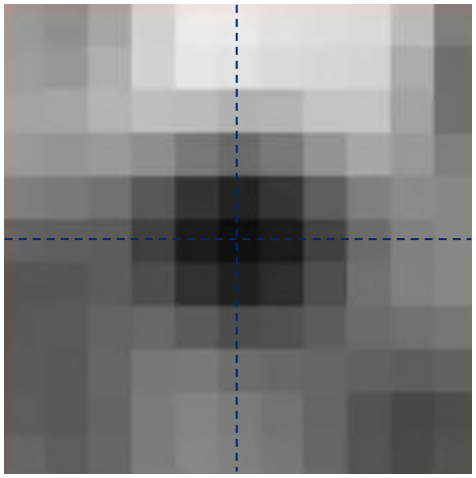
$I(x, y)$



y

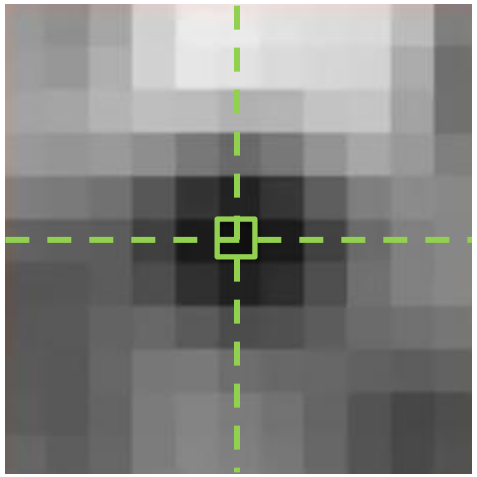
x

$E(u, v)$ v

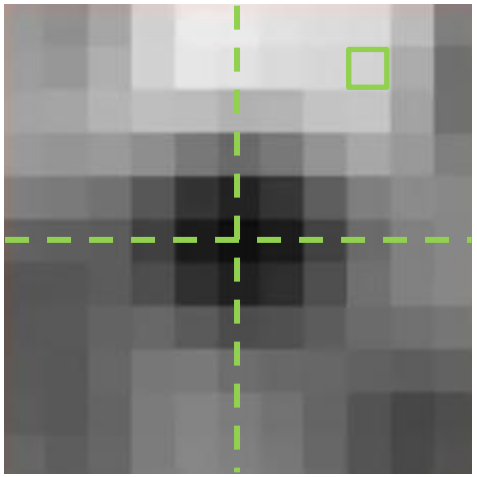


u

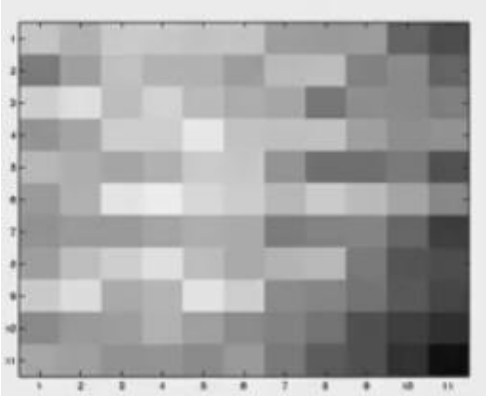
$E(0, 0)$

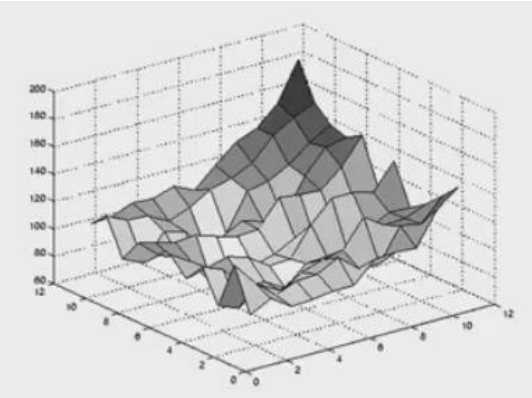


$E(3, 4)$

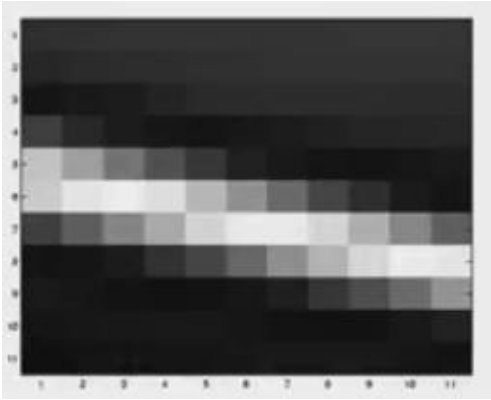


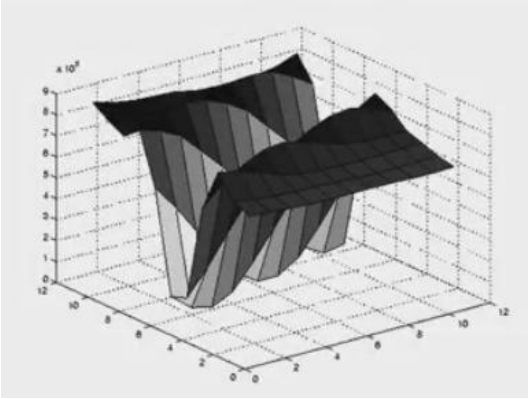

50
100
150
200
50
100
150
200
250
300
350



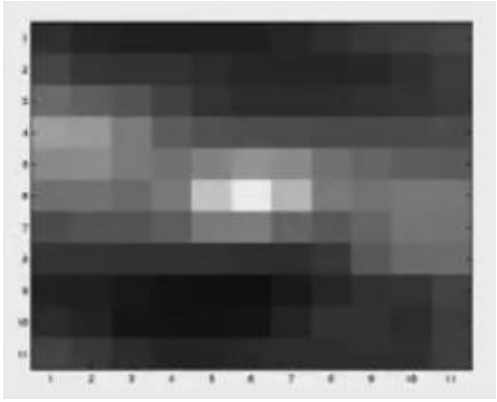


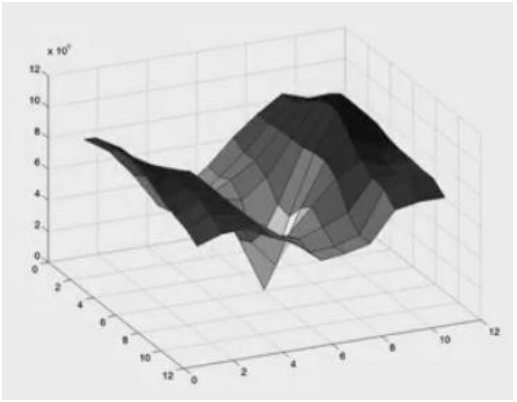

50
100
150
200
50
100
150
200
250
300
350






50
100
150
200
50
100
150
200
250
300
350





Harris corner detector: formulazione matematica (1)

∇_x: gradiente di I in *x*

∇_y: gradiente di I in *y*

Utilizzando l'espansione di Taylor del secondo ordine di *E(u,v)* la funzione precedente diventa:

$$E(u,v) = \sum_{x,y} w(x,y)[I(x+u, y+v) - I(x,y)]^2$$

$$\approx \sum_{x,y} w(x,y)\left[\nabla_x(x,y)u + \nabla_y(x,y)v + O(u^2, v^2)\right]^2$$

$$E(u,v) \approx \sum_{x,y} w(x,y)\nabla_x^2(x,y)u^2 + 2w(x,y)\nabla_x(x,y)\nabla_y(x,y)uv + w(x,y)\nabla_y^2(x,y)v^2$$

$$A = \sum_{x,y} w(x,y)\nabla_x^2(x,y),\ B = \sum_{x,y} w(x,y)\nabla_y^2(x,y),\ C = \sum_{x,y} w(x,y)\nabla_x(x,y)\nabla_y(x,y)$$

$$E(u,v) = Au^2 + 2Cuv + Bv^2$$

$$\Rightarrow E(u,v) = \begin{bmatrix} u & v \end{bmatrix} \begin{bmatrix} A & C \\ C & B \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}$$

Harris corner detector: formulazione matematica (2)

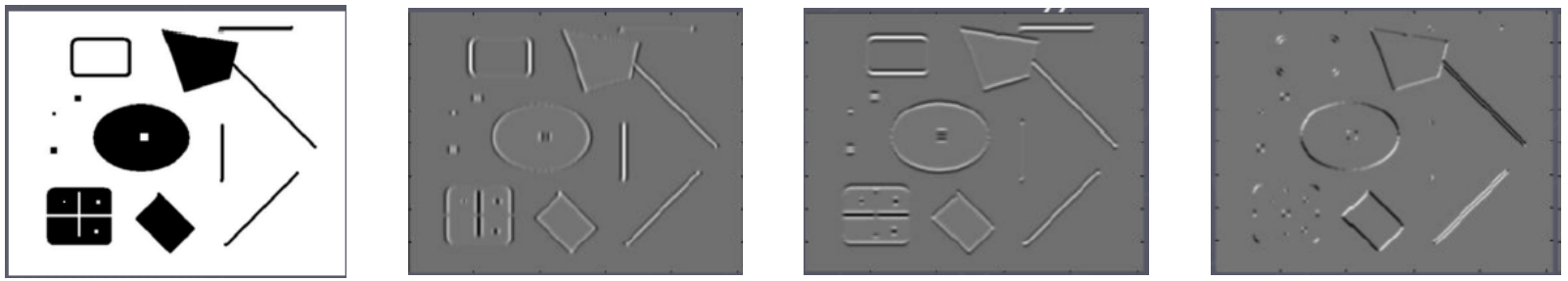
Per piccoli spostamenti $[u, v]$ la funzione può essere approssimata con:

$$E(u,v) \approx [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

dove *M* è una matrice 2×2 calcolata a partire dal gradiente dell'immagine:

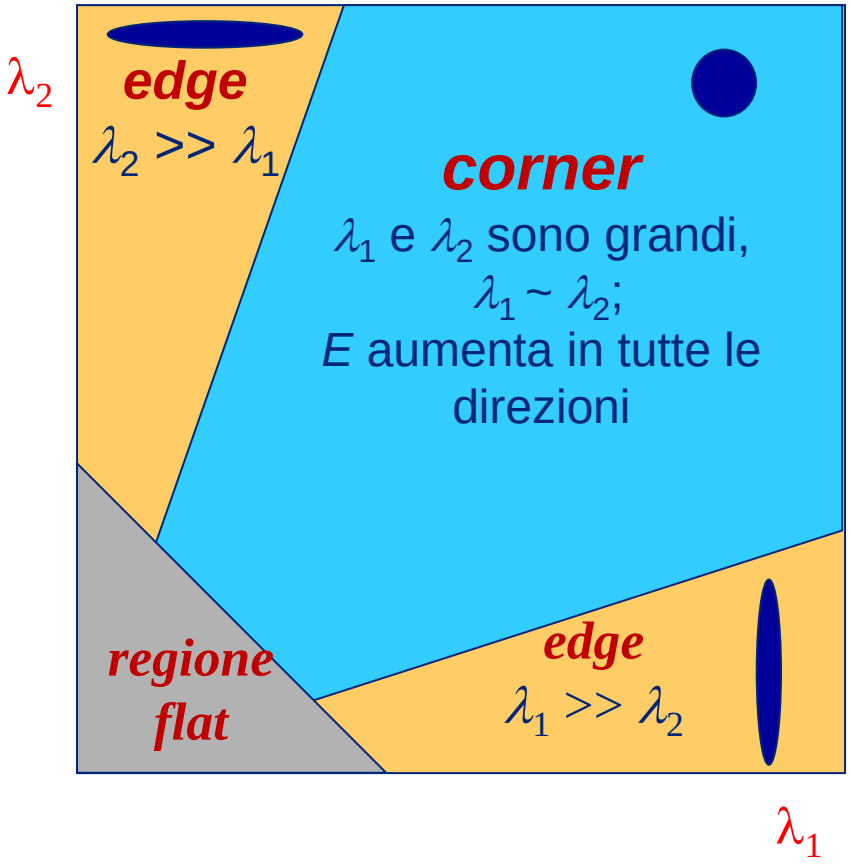
$$M = \sum_{x,y} w(x,y) \begin{bmatrix} \nabla_x^2(x,y) & \nabla_x(x,y)\nabla_y(x,y) \\ \nabla_x(x,y)\nabla_y(x,y) & \nabla_y^2(x,y) \end{bmatrix}$$

I ∇_x^2 ∇_y^2 $\nabla_x \nabla_y$



Harris corner detector: formulazione matematica (3)

- Un'analisi degli autovalori λ_1, λ_2 della matrice *M* permette di rilevare la presenza di corner.



λ_1 e λ_2 sono piccoli; *E* è pressoché costante in tutte le direzioni.

Harris corner detector: formulazione matematica (4)

- Nella pratica la ricerca dei corner si basa sull'analisi del coefficiente di cornerness:

$$R = \lambda_1\lambda_2 - \alpha(\lambda_1 + \lambda_2)^2 = \det(M) - \alpha\, trace(M)^2$$

dove α è una costante che normalmente assume valori in $[0.04, 0.06]$.

N.B. Il calcolo di R non richiede il calcolo esplicito di λ_1 e λ_2

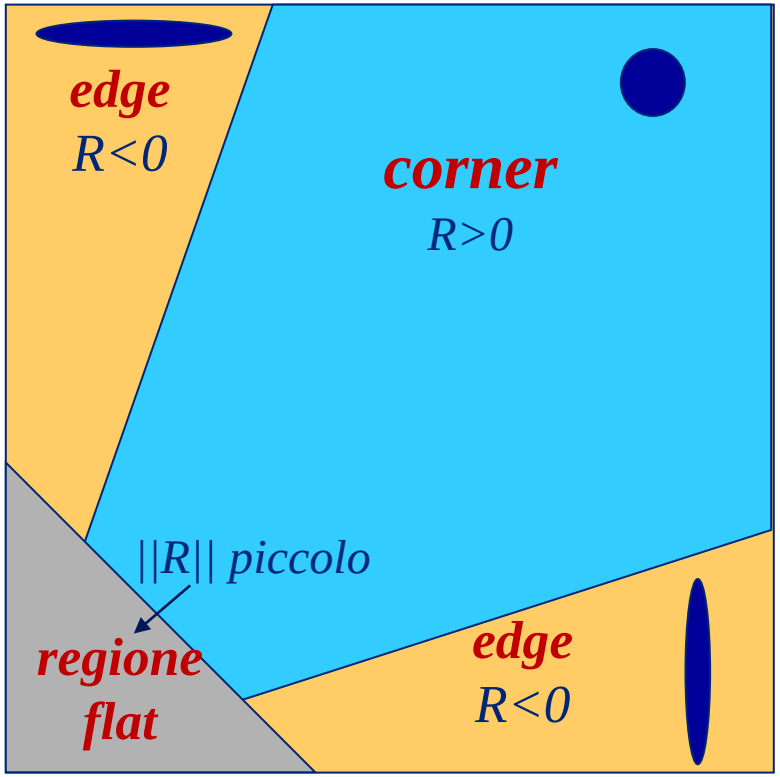
- Se $R < 0$ si ha un edge, se $R > 0$ si ha un corner, se $|R|$ è piccolo si è in presenza di una zona flat.

Cornerness

$$R = \det(M) - \alpha\, trace(M)^2 = \lambda_1\lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$$

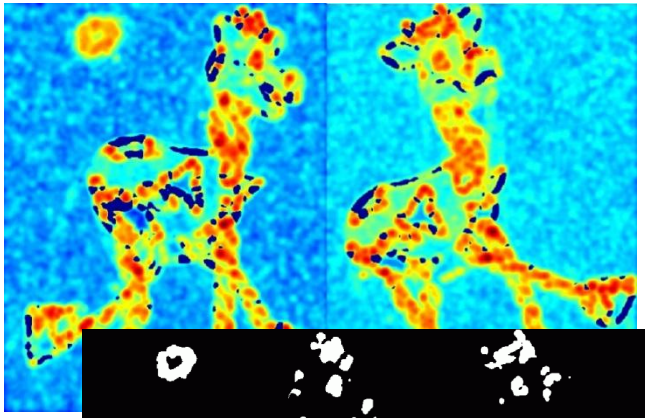
R assumerà:

- valori grandi nei corner;
- Valori piccoli nelle regioni flat
- Valori negativi in corrispondenza di edge

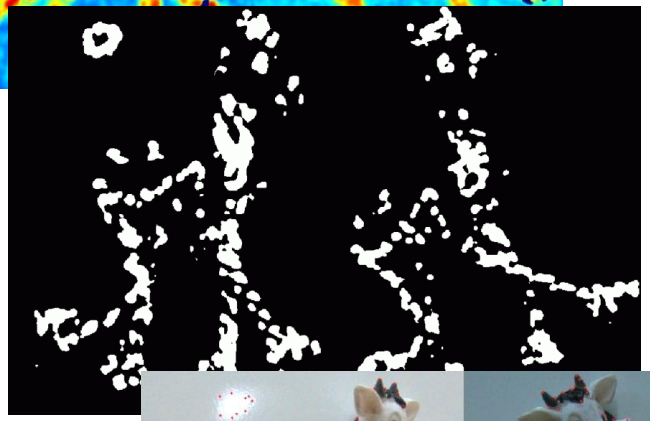


Harris corner detector: algoritmo

1. Calcolo del valore di *cornerness* (valore *R*) per ciascun pixel dell'immagine;



2. Identificazione dei punti per i quali le corrispondenti finestre ottengono score elevati *(R > soglia)*;



3. Selezione dei punti di massimo locale (soppressione dei non massimi).

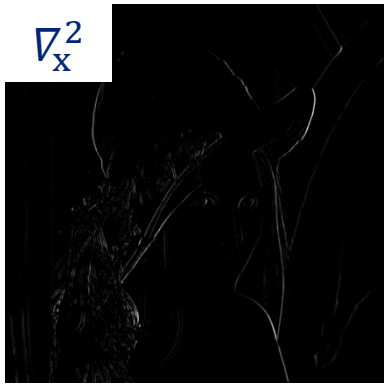


I



Gradient

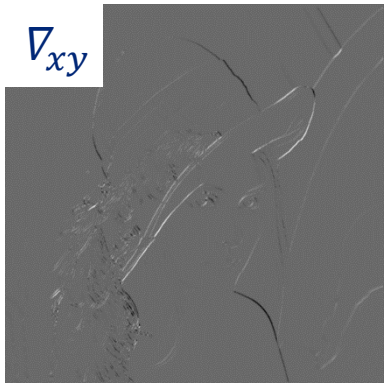
∇_x^2



∇_y^2



∇_{xy}



Gaussian blurring

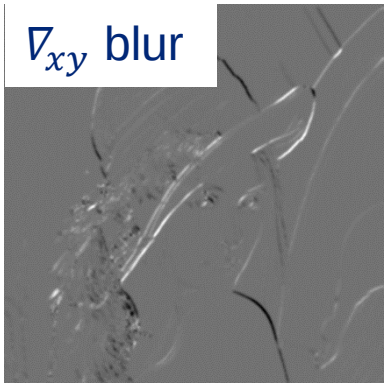
∇_x^2 blur



∇_y^2 blur



∇_{xy} blur



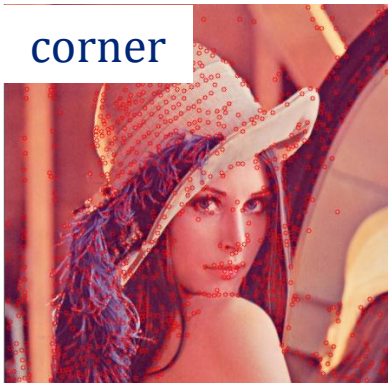
Cornerness map

cornerness



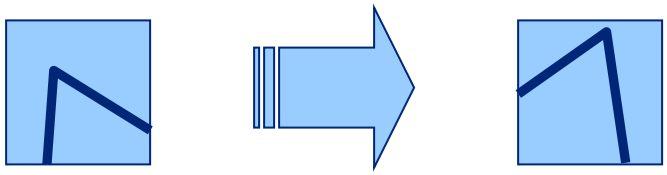
Soppressione non massimi

corner

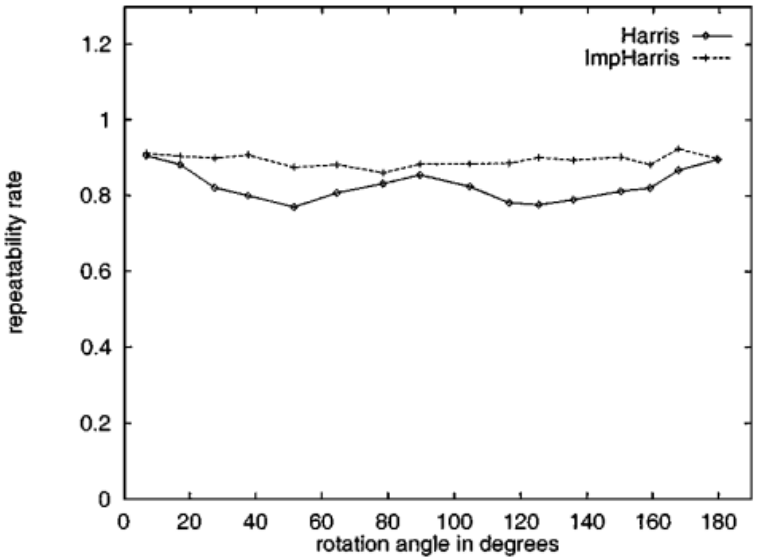


Harris corner detector: proprietà (1)

Invarianza per rotazione



$$RR = \frac{\#correspondences\ found}{\#possible\ correspondences}$$

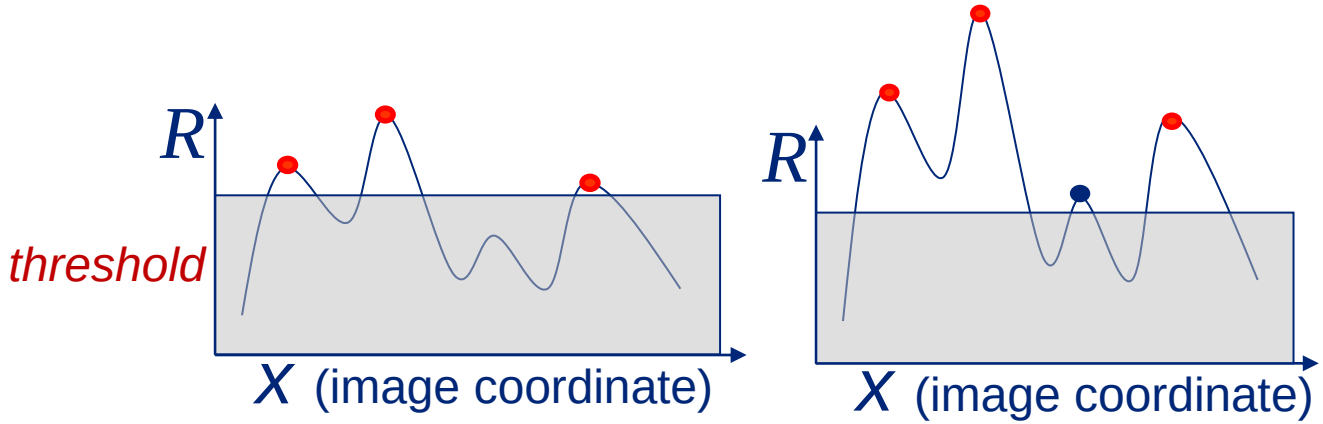


Harris corner detector: proprietà (2)

Invarianza parziale a cambiamenti di intensità:

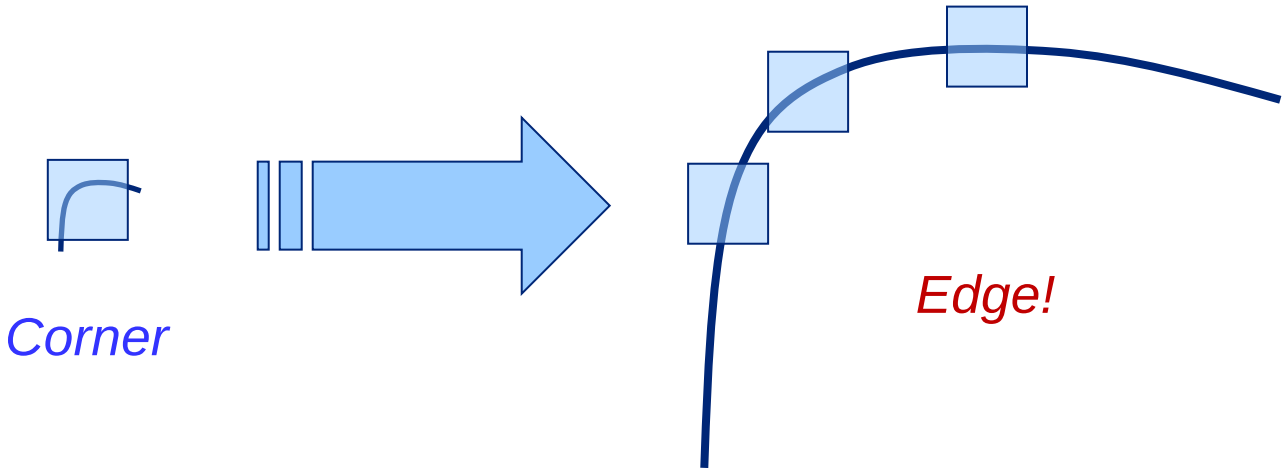
$$I \Rightarrow I + b$$

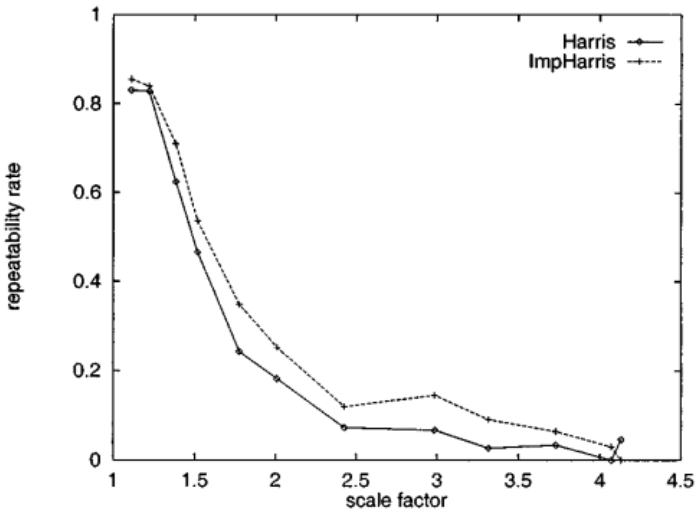
$$I \Rightarrow aI$$



Harris corner detector: proprietà (3)

Mancata invarianza rispetto a cambiamenti di scala





Localizzazione invariante per cambiamenti di scala

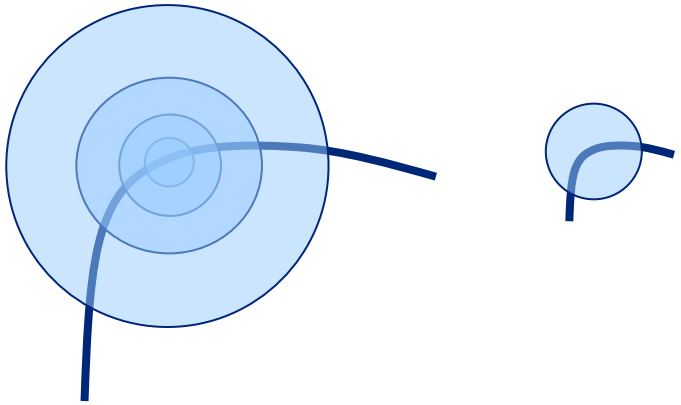
L'obiettivo è quello di individuare punti di interesse invarianti rispetto a cambiamenti di scala (e prospettiva…).

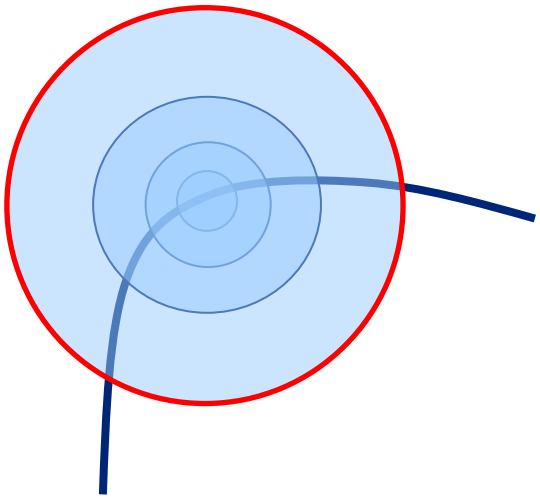


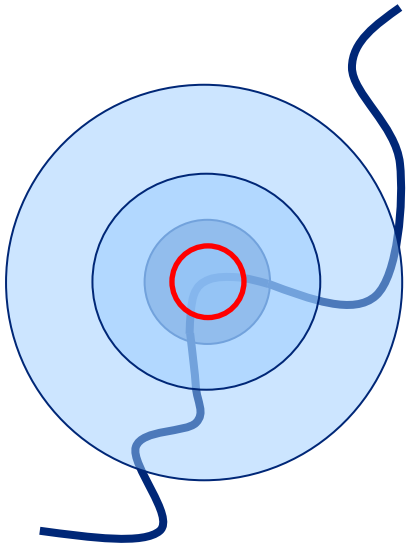


Localizzazione invariante per cambiamenti di scala

Date due immagini con diversa scala, consideriamo delle regioni (es. circolari) di diverse dimensioni centrate su un punto di interesse. Regioni di dimensioni corrispondenti avranno un aspetto simile nelle due immagini. Il problema è quello di trovare regioni di interesse in modo indipendente nelle due immagini.

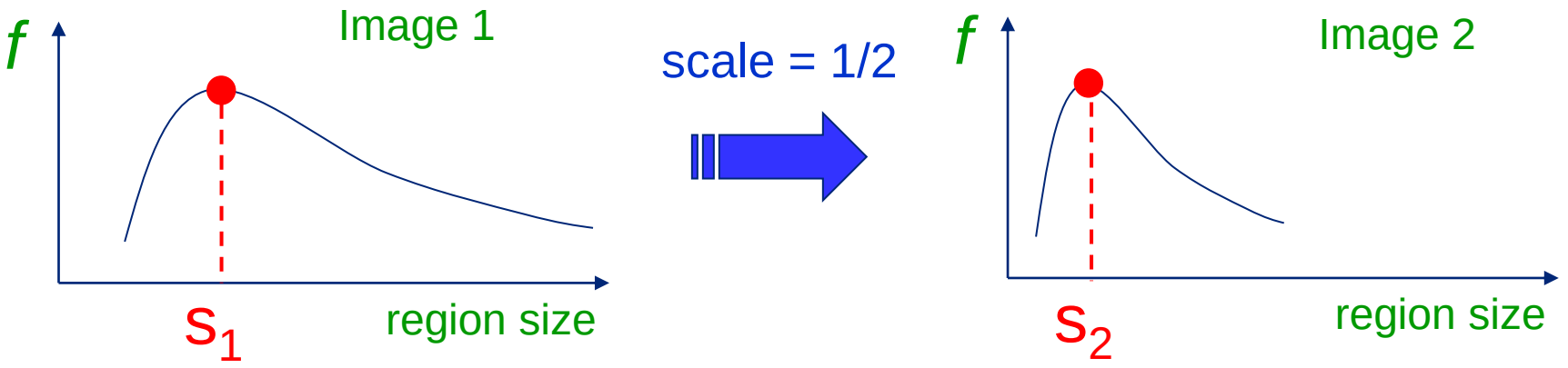






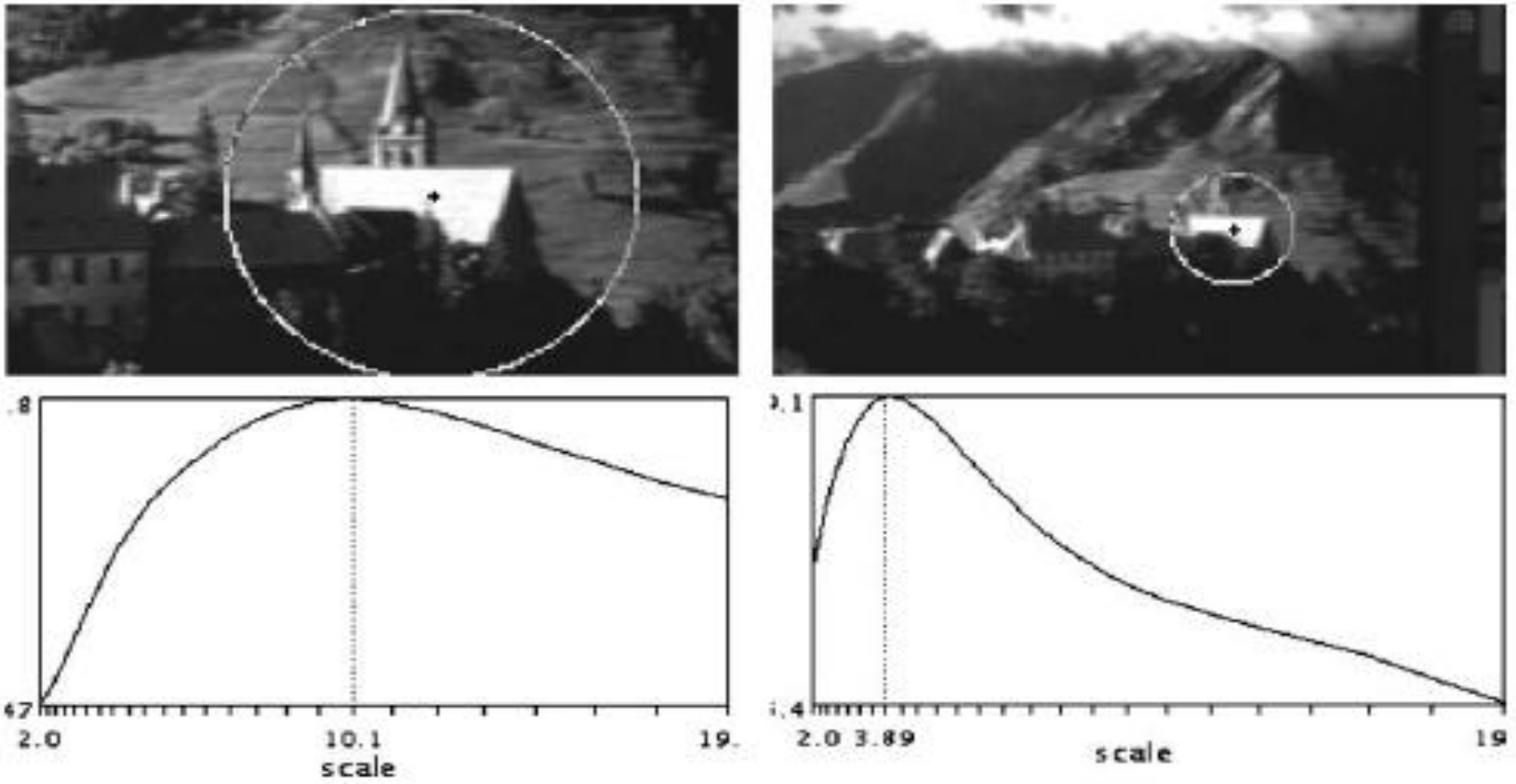
Localizzazione invariante per cambiamenti di scala: idea

- Una possibile soluzione al problema è data dalla definizione (sulla regione) di una funzione invariante per cambiamenti di scala (es. intensità media).



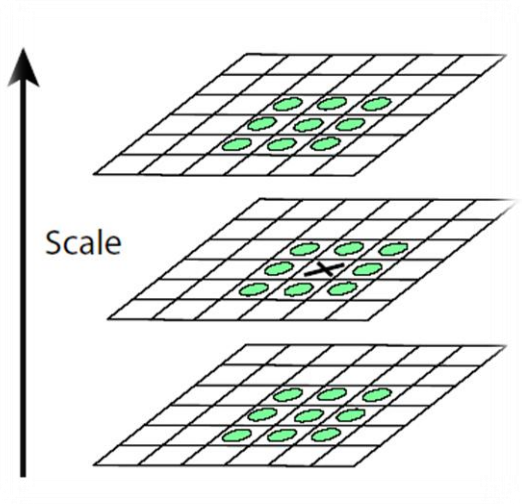
L'approccio comune consiste nel prendere i punti di massimo di questa funzione. Il rapporto tra le due scale in corrispondenza delle quali si osserva il valore massimo corrisponde al fattore di scala tra le due immagini.

Localizzazione invariante per cambiamenti di scala: idea



Lo scale space





- Una rappresentazione multi-scala di un'immagine si ottiene applicando filtri gaussiani caratterizzati da valori crescenti di σ.
- L'effetto visivo prodotto è quello di una progressiva e crescente sfuocatura dell'immagine.

Gaussiana, derivata e laplaciano (1)

- La rappresentazione multi-risoluzione di immagini digitali più utilizzata è basata sull'uso di filtri Gaussiani. La funzione Gaussiana bidimensionale è definita come:

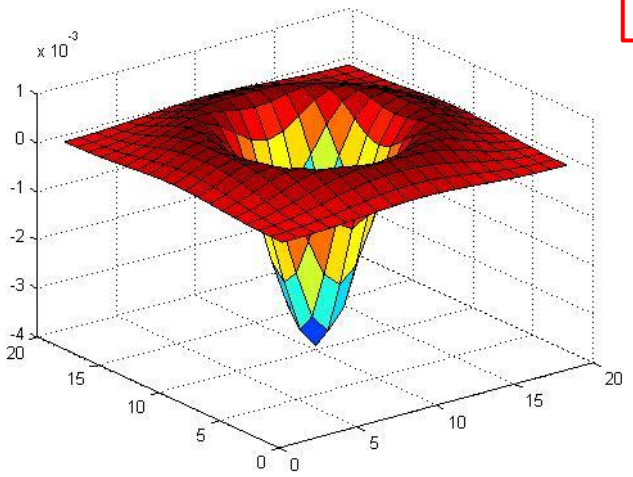
$$G(\sigma) = \frac{1}{2\pi\sigma^2}\, exp^{-\frac{x^2+y^2}{2\sigma^2}}$$

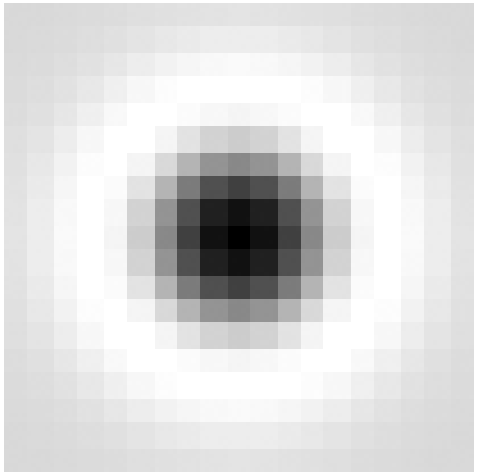
- Il filtro Gaussiano è un filtro separabile, ovvero il filtraggio di un segnale bidimensionale può essere sostituito da due operazioni di filtraggio monodimensionali (uno per ciascuna dimensione); ciò rende l'operazione più efficiente.
- È possibile ottenere rappresentazioni dell'immagine a diverse scale operando convoluzioni con opportuni kernel Gaussiani.

Gaussiana, derivata e laplaciano (2)

Alcune particolari strutture caratteristiche (edge, blob, ecc.) possono essere individuate all'interno dell'immagine ricorrendo a derivate della funzione Gaussiana. Una delle funzioni più interessanti per la localizzazione di blob è detta Laplacian of Gaussian e corrisponde alla derivata seconda della funzione Gaussiana:

$$LoG = \frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2}$$





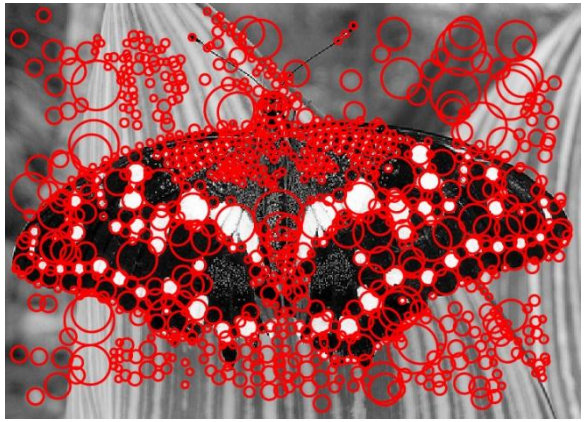
Laplacian of Gaussian detector

Il Laplacian of Gaussian detector proposto in [Lind98] prevede i seguenti passaggi:

1. Convoluzione dell'immagine con Laplaciano normalizzato a diverse scale;



2. Ricerca dei massimi nello *scale-space* della risposta del Laplaciano al quadrato.





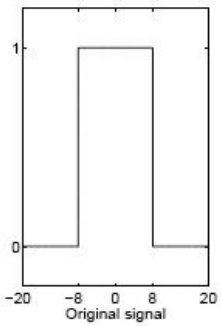
sigma = 9.5859

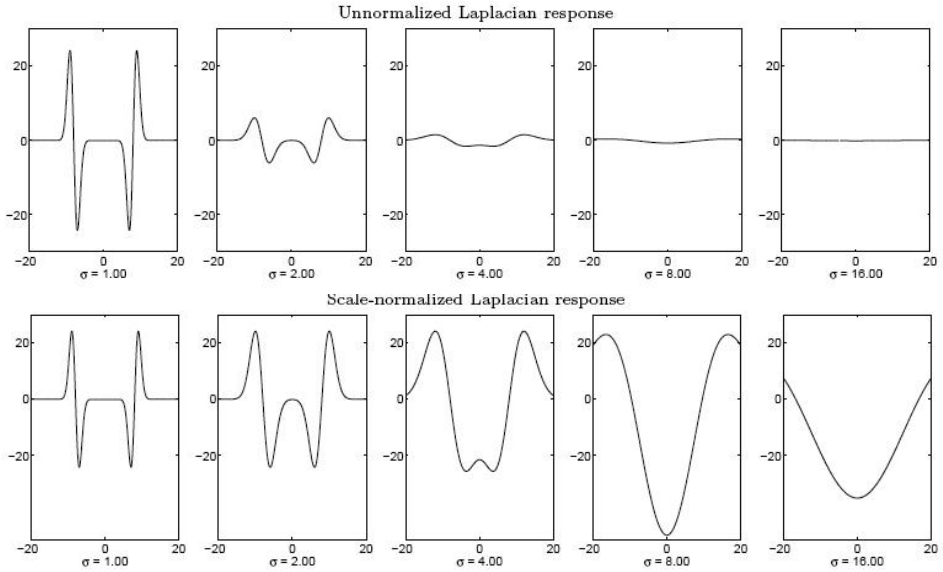
Gaussiana, derivata e laplaciano (3)

Nell'analisi multi-risoluzione di un'immagine bisogna considerare che la risposta del Laplaciano diminuisce all'aumentare della scala. Per rendere la risposta invariante per scala è necessario normalizzare la funzione effettuando una moltiplicazione per σ^2.

$$LoG_{norm}(x,y) = \sigma^2\left(\frac{\partial^2 G}{\partial x^2} + \frac{\partial^2 G}{\partial y^2}\right)$$

Original signal



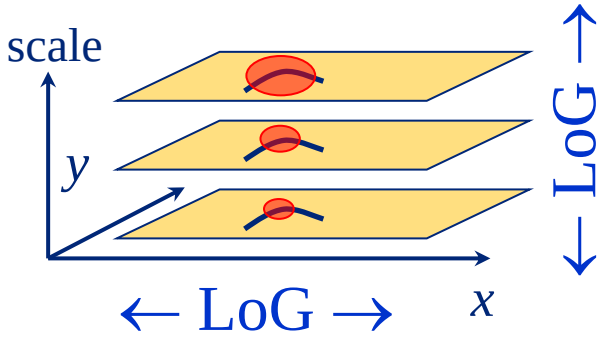


Unnormalized Laplacian response

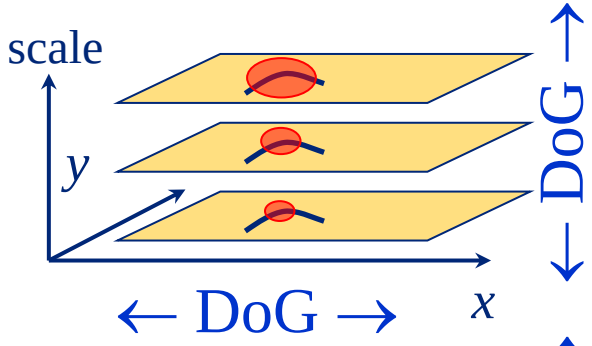
Scale-normalized Laplacian response

Localizzatori invarianti per scala

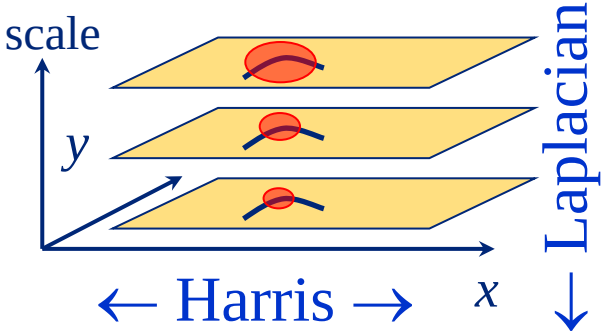
- **Laplacian detector** [Lind98]: è uno dei più noti blob detector; effettua una convoluzione dell'immagine con il Laplaciano a diverse scale e cerca successivamente massimi e minimi nello *scale space*.



- **Difference of Gaussian** [Lowe'99]: approssimazione del Laplacian of Gaussian per un calcolo più efficiente.



- **Harris-Laplace** [Miko01]: localizzazione dei corner con l'operatore di Harris a diverse scale e la successiva selezione dei punti corrispondenti a *massimi del Laplaciano* sulle diverse scale.



Scale Invariant Feature Transform (SIFT)

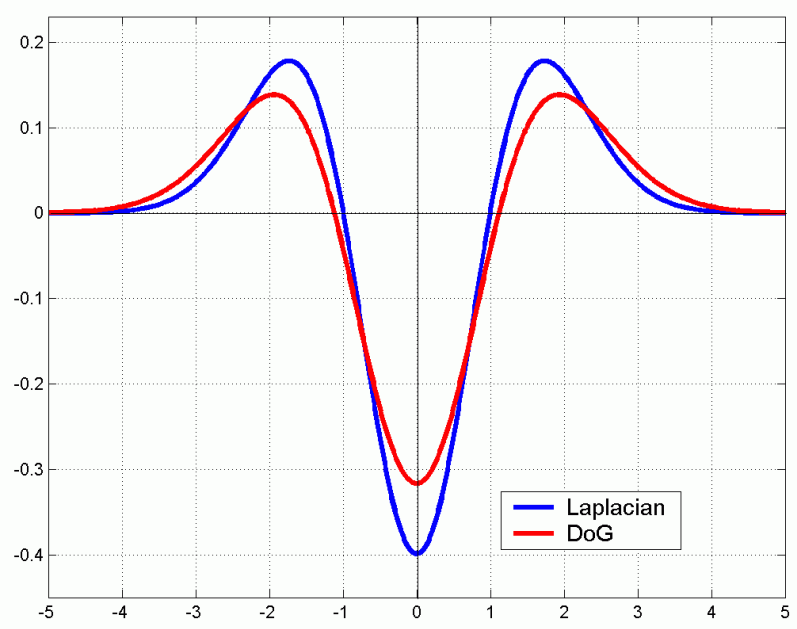
L'approccio proposto in [Lowe99] per l'estrazione di punti salienti (detti keypoint) si basa su un'approssimazione del Laplacian of Gaussian denominata Difference of Gaussian.

$$LoG = \sigma^2 \left(G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma) \right)$$

(Laplacian)

$$DoG = G(x, y, k\sigma) - G(x, y, \sigma)$$

(Difference of Gaussians)



Difference of Gaussian

Per la localizzazione dei punti di interesse viene costruita una rappresentazione multi-risoluzione dell'immagine tramite convoluzione della stessa con kernel Gaussiani di diverse scale:

$$L(x, y, k\sigma) = G(x, y, k\sigma) * I(x, y)$$

dove $G(x, y, k\sigma)$ è la Gaussiana con scala variabile e $I(x, y)$ è l'immagine di input.

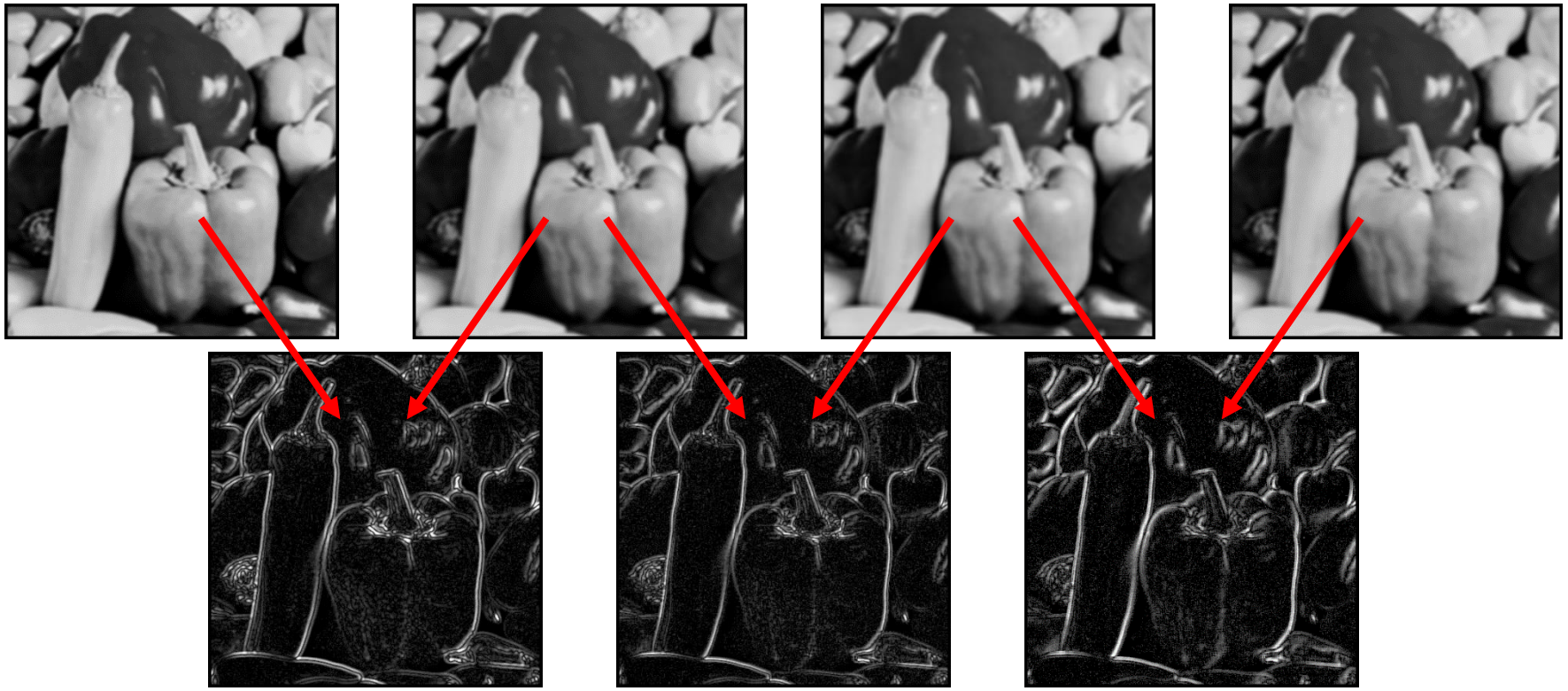





Difference of Gaussian

- La localizzazione dei punti avviene ricercando i punti estremi rispetto alla funzione *Difference of Gaussian*:

$$D(x, y, \sigma) = \big(G(x, y, k\sigma) - G(x, y, \sigma)\big) * I(x, y)$$
$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$



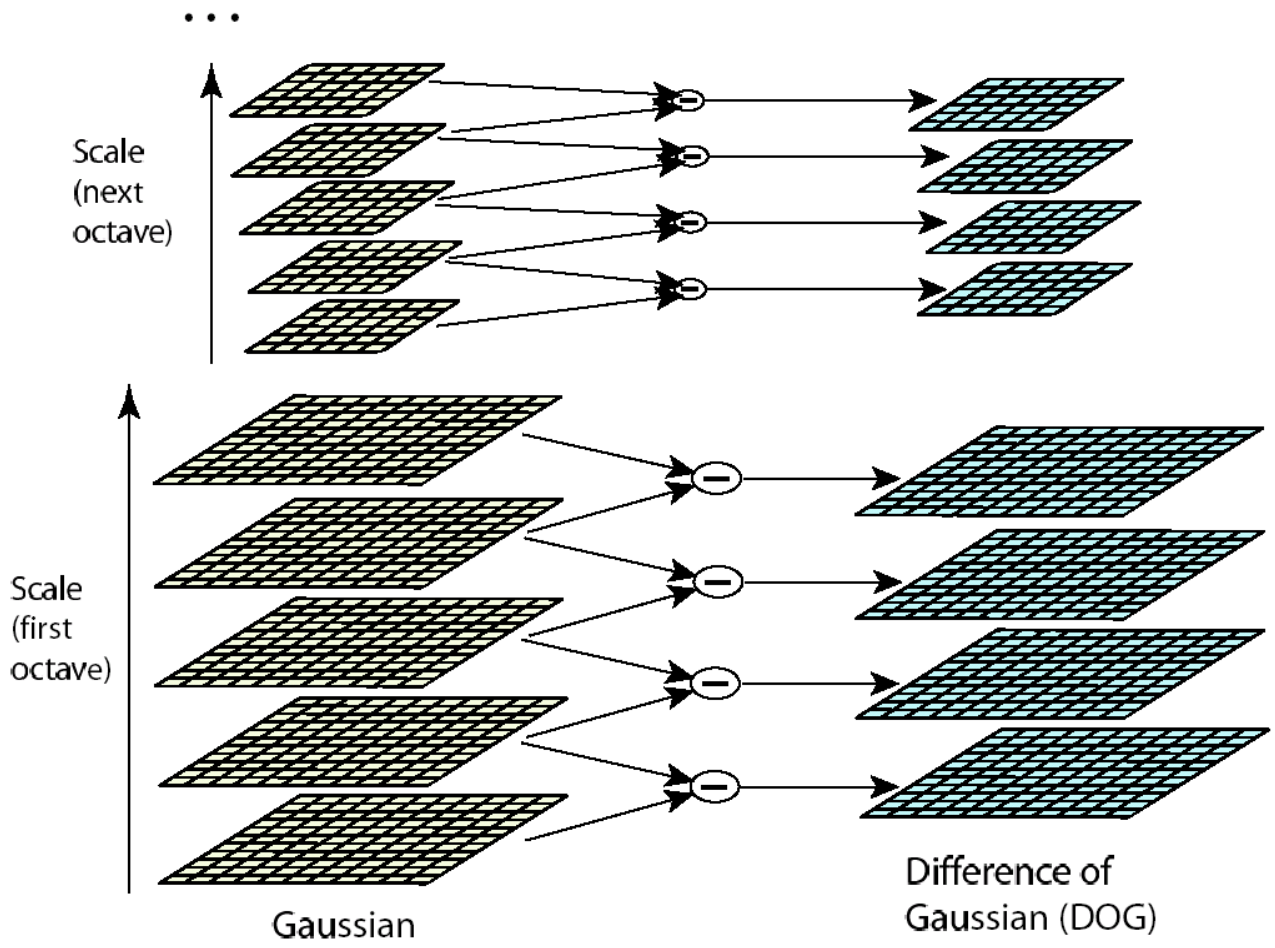
DoG: motivazioni (1)

- L’utilizzo della differenza di Gaussiane rende la localizzazione più efficiente in quanto le immagini smoothed (convoluzione con Gaussiana) sono comunque necessarie per il calcolo dei descrittori associati ai punti, quindi la localizzazione richiede in aggiunta delle semplici operazioni di sottrazione.
- La funzione DoG è inoltre un’ottima approssimazione della funzione LoG (necessaria per raggiungere una vera invarianza per scala).

DoG e localizzazione (1)

- Il calcolo dello scale-space e relative *DoG* è computazionalmente oneroso; per rendere il calcolo più efficiente si utilizza una struttura piramidale che sfrutta la relazione $L(x, y, 2\sigma) = L'(x, y, \sigma)$ dove *L'* è un livello dello scale-space a metà risoluzione rispetto a *L*.
- Lo scale-space può dunque essere diviso in ottave, ciascuna costituita da *mappe* ottenute tramite convoluzione dell'immagine con filtri di scala variabile nel range $[\sigma, 2\sigma]$.

DoG e localizzazione (2)

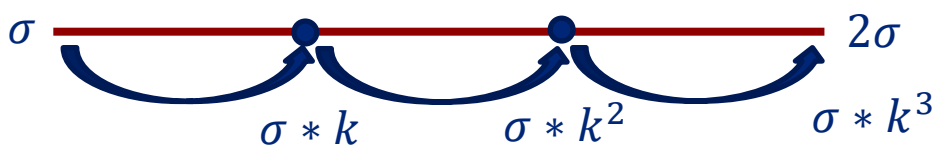


DoG e localizzazione (1)

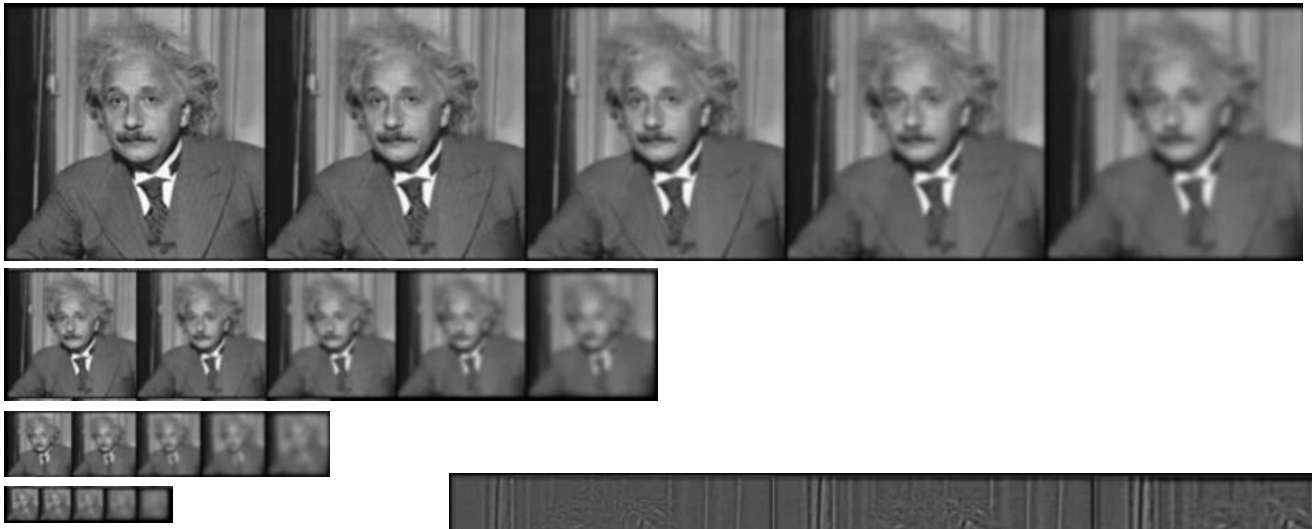
- Il numero di livelli per ogni ottava è regolato dal parametro ***s*** che indica il numero di intervalli in cui è diviso lo scale-space. Il valore di ***k*** che rappresenta il moltiplicatore di scala rispetto alla scala base, risulta di conseguenza pari a $k = 2^{\frac{1}{s}}$.

$s = 3$

$k = 2^{\frac{1}{3}}$



DoG e localizzazione: un esempio



Gaussian

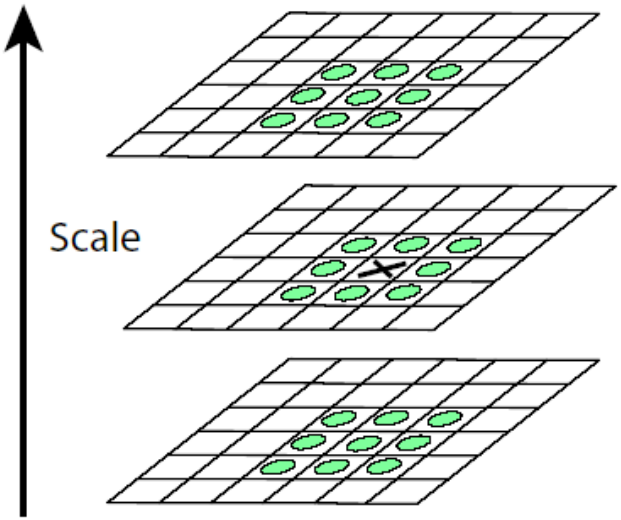
Difference of Gaussian



Localizzazione approssimata

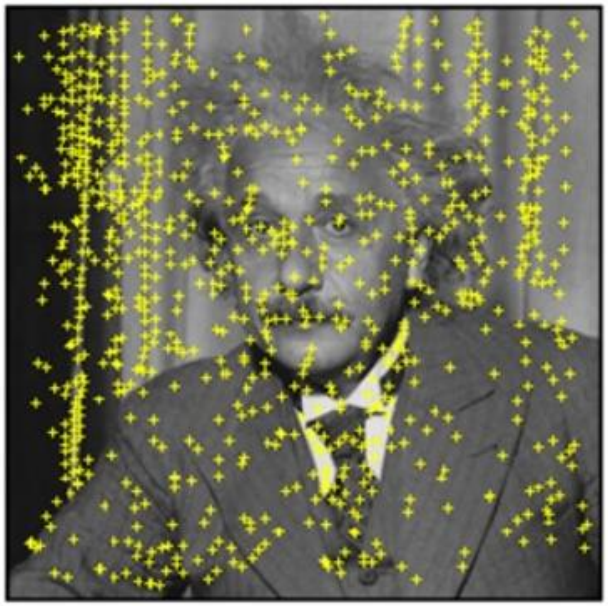
- La mappa di un determinato livello, grazie alla proprietà piramidale, presenta scala doppia rispetto alla mappa dello stesso livello nell'ottava inferiore.
- Al fine di individuare i punti su un'intera ottava è necessario calcolare *s*+3 livelli relativi a convoluzioni gaussiane successive.

- Una volta calcolate tutte le immagini *DoG* i punti di interesse possono essere individuati come punti estremi (massimi e minimi) delle differenze calcolate.
- Per essere considerato estremo, un pixel di una DoG deve essere maggiore o minore di tutti i suoi 8 vicini (della stessa immagine) e di tutti i vicini (9 + 9) relativi all'immagine della scala superiore e di quella inferiore.

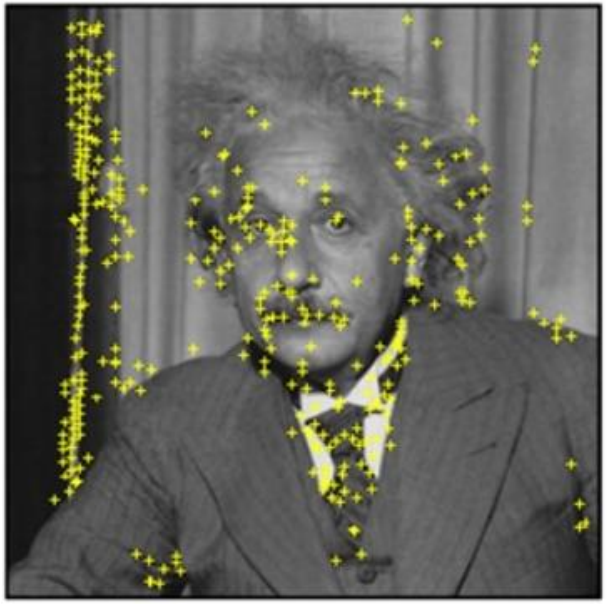


Localizzazione accurata

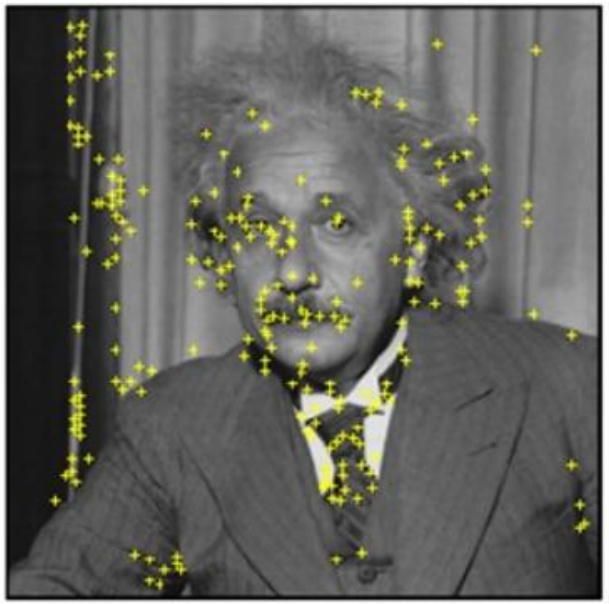
- I punti individuati nello scale-space sono localizzati in corrispondenza dei pixel che, essendo per loro natura discreti, non rappresentano esattamente il punto estremo dell'intorno.
- Per localizzare ciascun punto in modo più accurato può essere effettuata un'interpolazione tra i valori dell'intorno.



Punti estremi

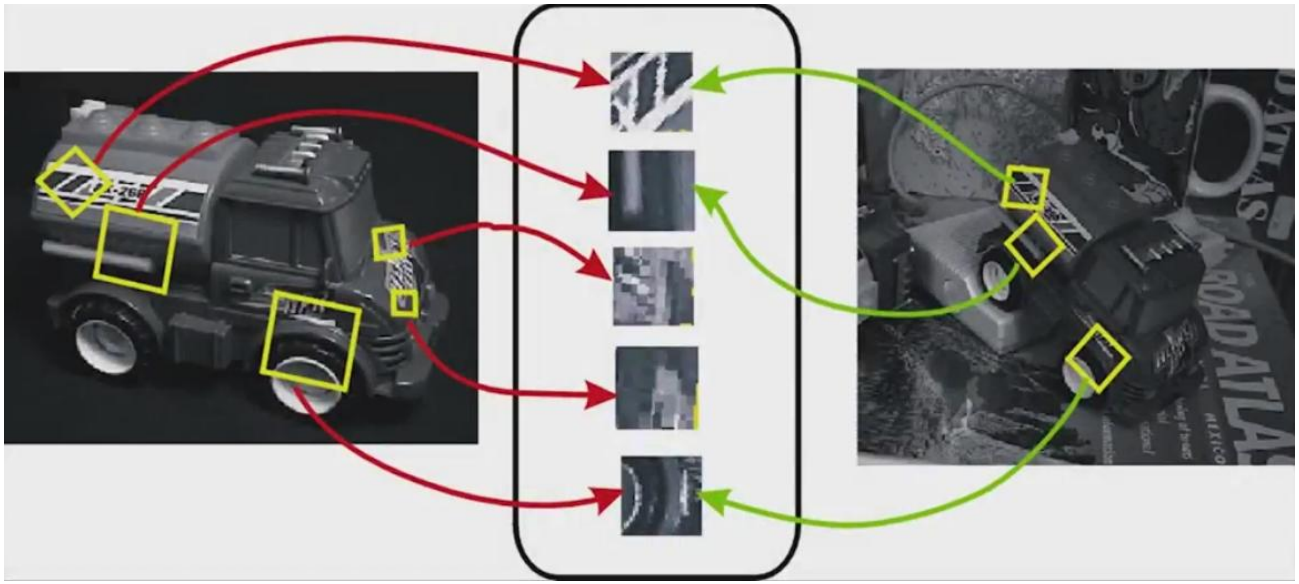


Soglia Contrasto



Non Edge

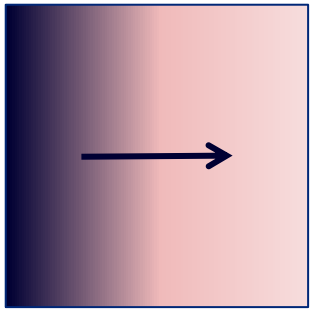
Matching: i descrittori

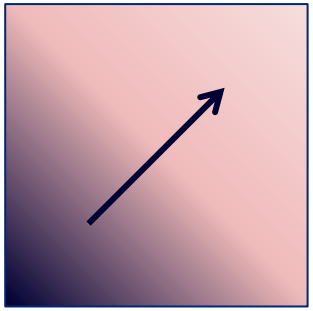






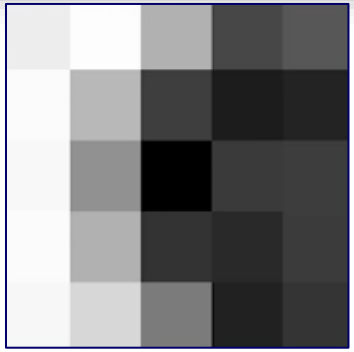
Orientazione dei keypoint

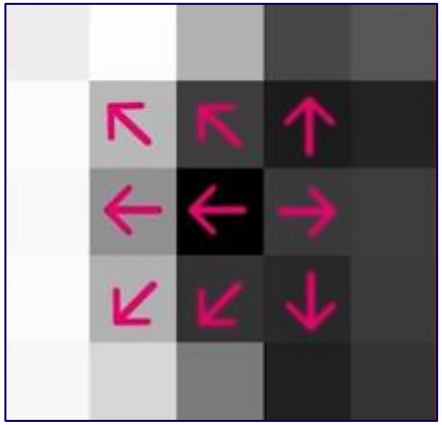


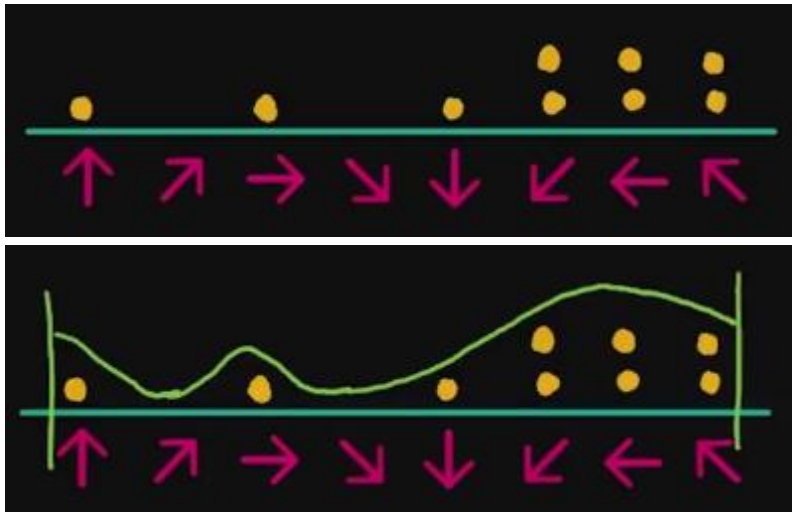


Idea
individuare la direzione con gradiente maggiore









Orientazione (1)

Al fine di costruire dei descrittori locali invarianti rispetto alla rotazione (planare) della scena, viene assegnata a ogni punto di interesse una orientazione.

Per il calcolo dell'orientazione si prende come riferimento l'immagine smoothed *L* caratterizzata dalla scala più vicina a quella del punto in esame. Si analizzano i punti della regione circostante e si calcolano magnitudo e orientazione del gradiente:

$$m(x,y) = \sqrt{\big(L(x+1,y) - L(x-1,y)\big)^2 + \big(L(x,y+1) - L(x,y-1)\big)^2}$$

$$\theta(x,y) = \tan^{-1}\big(L(x,y+1) - L(x,y-1)\big) / \big(L(x+1,y) - L(x-1,y)\big)$$

Orientazione (2)

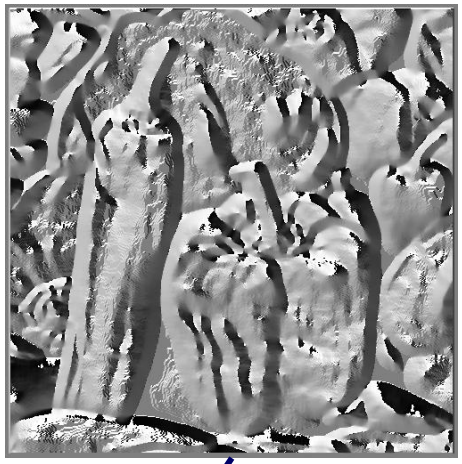
L



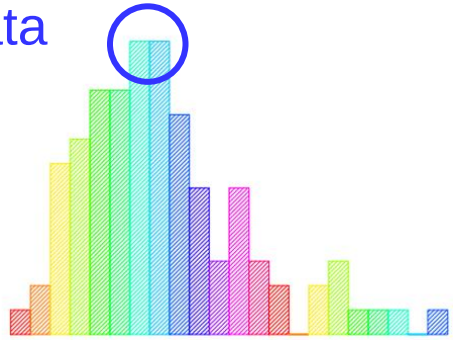
magnitudo



orientazioni



Orientazione assegnata



Istogramma orientazioni (36-bin)

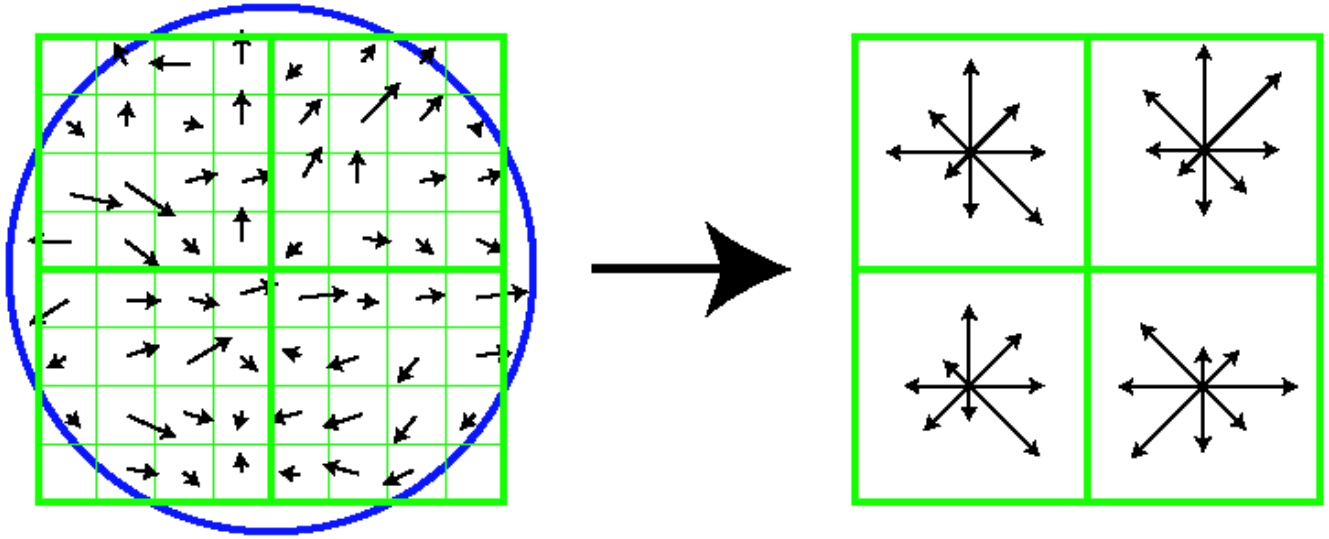
Orientazione (3)

- Considerando i pixel di una regione nell'intorno del punto corrente viene costruito un istogramma delle orientazioni del gradiente. L'istogramma è costituito da 36 bin, ciascuno dei quali copre un range di 10°.
- Ciascun pixel contribuisce ad aumentare il bin relativo alla sua orientazione di un valore pari al suo valore di magnitudo, pesato con funzione gaussiana ($\sigma = 1.5$ volte la scala del punto) centrata nel punto stesso.
- Il massimo assoluto dell'istogramma rappresenta l'orientazione da assegnare al punto.
- Se esistono uno o più massimi locali che superano l'80% del massimo assoluto, vengono generati nuovi keypoint con tali orientazioni (e stessa posizione e scala del punto originale).

Descrittori locali SIFT (1)

- Una volta individuati i punti di interesse è possibile associare a ciascuno un descrittore locale, un vettore di feature (valori numerici) che riassume le caratteristiche salienti di un intorno del punto. Sono i descrittori che poi saranno usati per le operazioni di classificazione/matching.
- I descrittori SIFT vengono calcolati sulla base delle informazioni relative a orientazione e magnitudo del gradiente in una finestra circostante il punto di interesse.
  - Il valore di scala del keypoint viene utilizzato per selezionare le mappe di orientazione e magnitudo (calcolate per l'assegnazione dell'orientazione), mentre il valore di orientazione serve per orientare la finestra.
  - I valori di magnitudo dei pixel della finestra vengono pesati con una funzione gaussiana con σ pari a metà della larghezza della finestra.

Descrittori locali SIFT (2)



- I punti della regione di interesse sono partizionati in sottoregioni attorno al punto stesso.
- A ciascuna regione è associato un istogramma di 8 bin contenenti le distribuzioni pesate della magnitudo delle relative orientazioni.

Descrittori locali SIFT (3)

- Per rendere il descrittore più robusto si usa un'interpolazione trilineare per distribuire i valori del gradiente di ciascun punto anche nei bin adiacenti.
- Viene effettuata infine una normalizzazione del vettore di feature per ottenere una migliore invarianza rispetto a possibili cambiamenti di illuminazione.
  - In una prima fase *tutto il vettore viene normalizzato a 1* per gestire variazioni di illuminazione lineari;
  - Per eliminare l'effetto di variazioni di illuminazione non lineari (es. saturazione della camera) viene applicata una *soglia ai valori del vettore*, ovvero si mantengono gli elementi che hanno una valore non superiore a 0.2 e successivamente si *normalizza nuovamente* il vettore a 1.
- I parametri suggeriti per il calcolo del descrittore sono 4×4 istogrammi, ciascuno da 8 bin; la dimensione del descrittore è dunque 128.



Speeded-Up Robust Features

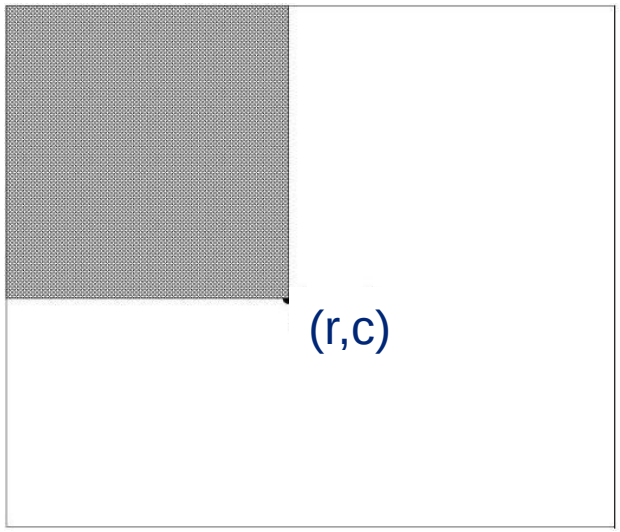
- La tecnica SURF (Speeded-Up Robust Features) è stata proposta in [Bay08] con l'obiettivo di localizzare e descrivere punti di interesse in modo estremamente efficiente, mantenendo al tempo stesso le proprietà di invarianza che caratterizzano la tecnica SIFT. I descrittori SURF risultano infatti invarianti rispetto a rotazioni dell'immagine, cambiamenti di scala e di illuminazione e a cambiamenti di prospettiva limitati.

- L'idea alla base del metodo è quella di utilizzare delle approssimazioni dei filtri usati tipicamente in altre tecniche per rendere il processo di analisi più efficiente. In particolare i tradizionali filtri LoG (Laplacian of Gaussian) vengono sostituiti da box filters che possono essere applicati in modo molto veloce alle immagini sfruttando il concetto di immagine integrale.

L'immagine integrale

L'***immagine integrale*** in posizione (*x*,*y*) è la somma del valore dei pixel sopra e a sinistra di (*x*,*y*):

$$II(r,c) = \sum_{r' \le r, c' \le c} I(r',c')$$



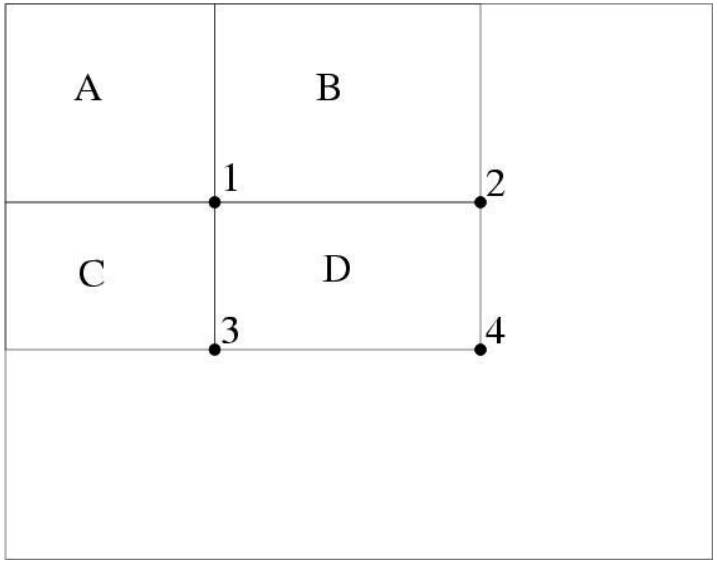
L'immagine integrale può essere calcolata con una sola scansione dell'immagine originale usando le seguenti formule ricorsive:

$$S(r,c) = S(r,c\text{-}1) + I(r,c)$$

$$II(r,c) = II(r\text{-}1,c) + S(r,c)$$

dove $I(r,c)$ è il valore del pixel nell'immagine originale, $S(r,c)$ è la somma cumulativa per righe,
$S(r,-1)=0$ e $II(-1,c)=0$.

Calcolo efficiente di feature rettangolari



Usando l'immagine integrale, è possibile calcolare la somma del valore dei pixel in qualsiasi rettangolo.

Ad esempio la somma all'interno del rettangolo D è data da:

$$II(4) + II(1) - II(2) - II(3)$$

Le feature caratterizzate da 2, 3 o 4 rettangoli possono essere calcolati usando rispettivamente 6, 8 e 9 valori di riferimento.

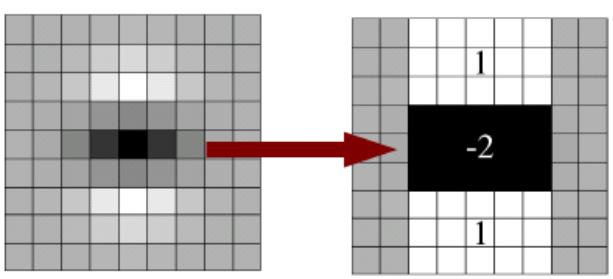
SURF: box filters (1)

La localizzazione dei punti di interesse si basa sull'analisi della matrice Hessiana:

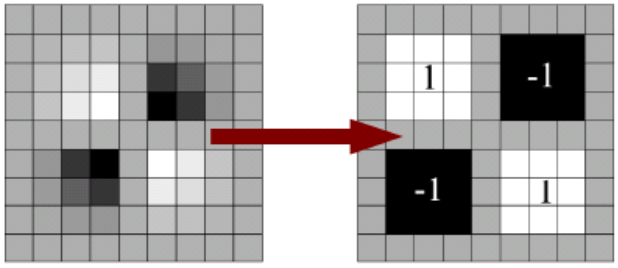
$$H = \begin{bmatrix} L_{xx} & L_{xy} \\ L_{xy} & L_{yy} \end{bmatrix}$$

dove $L_{xx}(x, y, \sigma)$ è il Laplacian of Gaussian dell'immagine, ovvero la convoluzione dell'immagine con la derivata seconda della funzione Gaussiana.

L'algoritmo di localizzazione SURF si basa su un'approssimazione della matrice Hessiana con dei box filter.



L_{yy}



L_{xy}

SURF: box filters (2)

- Il vantaggio nell'uso dei box filter è una notevole riduzione della complessità computazionale; questi filtri possono infatti essere applicati in modo molto efficiente ricorrendo all'immagine integrale.
- I box filter in figura rappresentano un'approssimazione di una Gaussiana con $\sigma = 1.2$ e rappresentano la scala più piccola (massima risoluzione). Nel seguito saranno indicati come D_{xx}, D_{yy} e D_{xy}.

SURF: localizzazione dei keypoint (1)

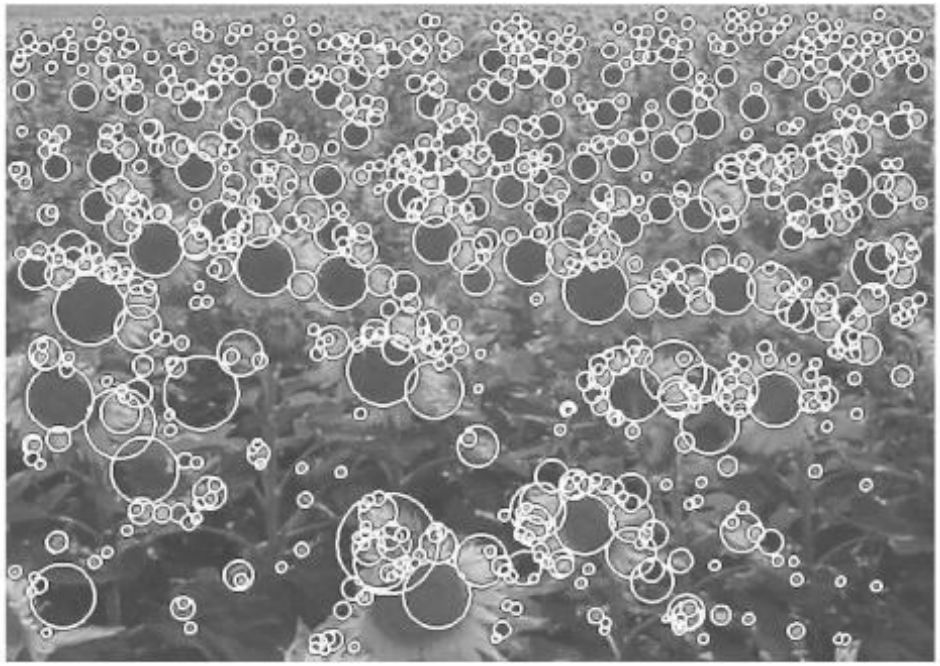
- La localizzazione dei punti di interesse (keypoint) si basa sul calcolo del determinante della matrice Hessiana approssimata:

$$det(H_{approx}) = D_{xx}D_{yy} - (wD_{xy})^2$$

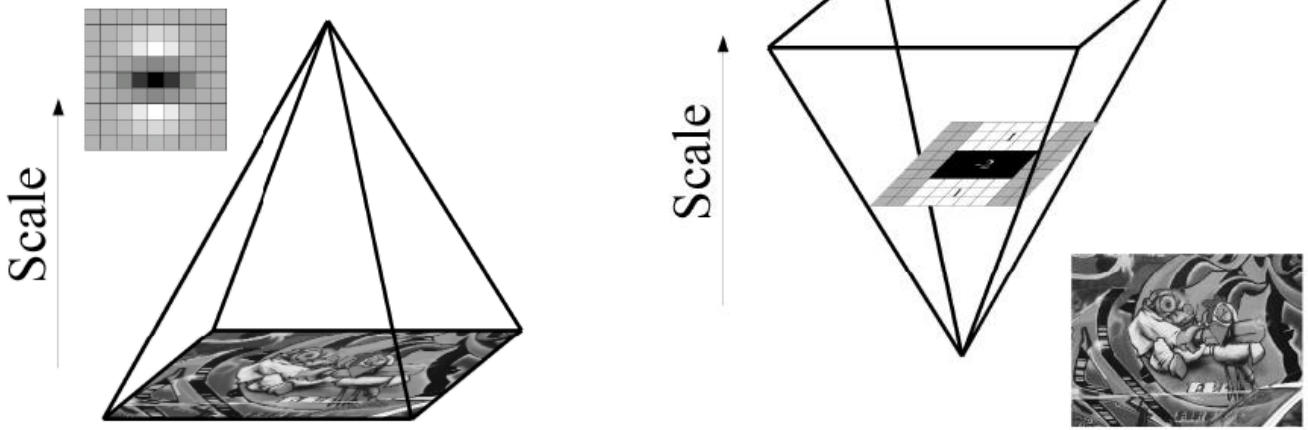
  dove *w* è un fattore di fissato al valore costante 0.9.
- Per approssimare meglio la gaussiana, la risposta dei filtri deve essere opportunamente pesata e normalizzata rispetto alla scala (dimensione del filtro).
- Il determinante della matrice Hessiana approssimata viene usato per determinare una mappa che viene calcolata per diversi valori di scala; i keypoint vengono selezionati in corrispondenza dei massimi locali della mappa.

SURF: localizzazione dei keypoint (2)

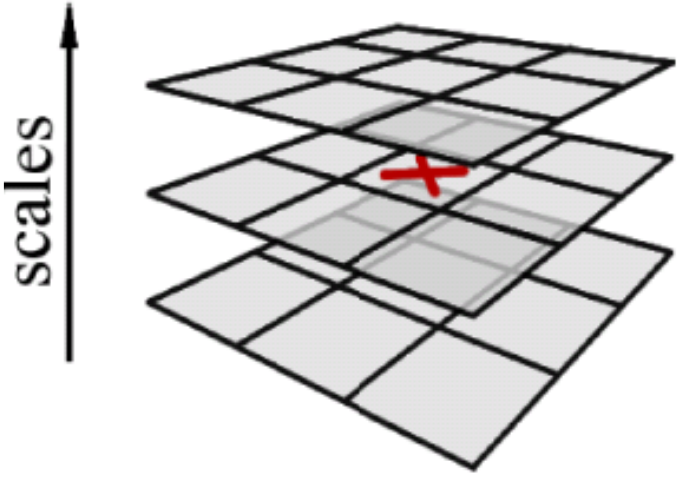
- A differenza di quanto avviene per le SIFT, in questo caso non è necessario costruire una piramide di immagini (con progressivi ridimensionamenti) in quanto l'applicazione di filtri di dimensioni diverse ha sempre la stessa complessità (e possono quindi essere applicati all'immagine originale).



SURF: localizzazione dei keypoint (3)

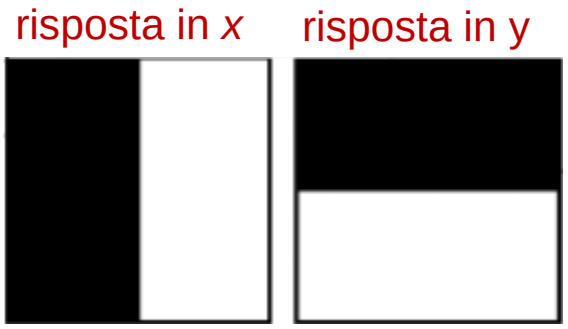


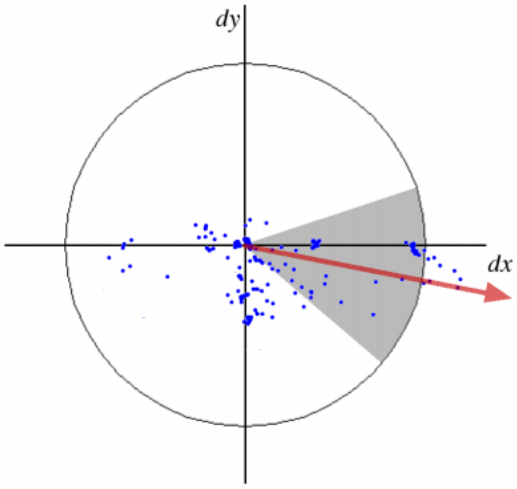
- Lo scale-space si ottiene ridimensionando i box-filters e viene diviso in ottave, ciascuna delle quali rappresenta una serie di mappe ottenute tramite convoluzione dell'immagine originale con filtri di dimensione crescente.
- Anche in questo caso la localizzazione dei punti prevede una soppressione dei non-massimi in un intorno 3D 3×3×3 e interpolazione.



SURF: orientazione

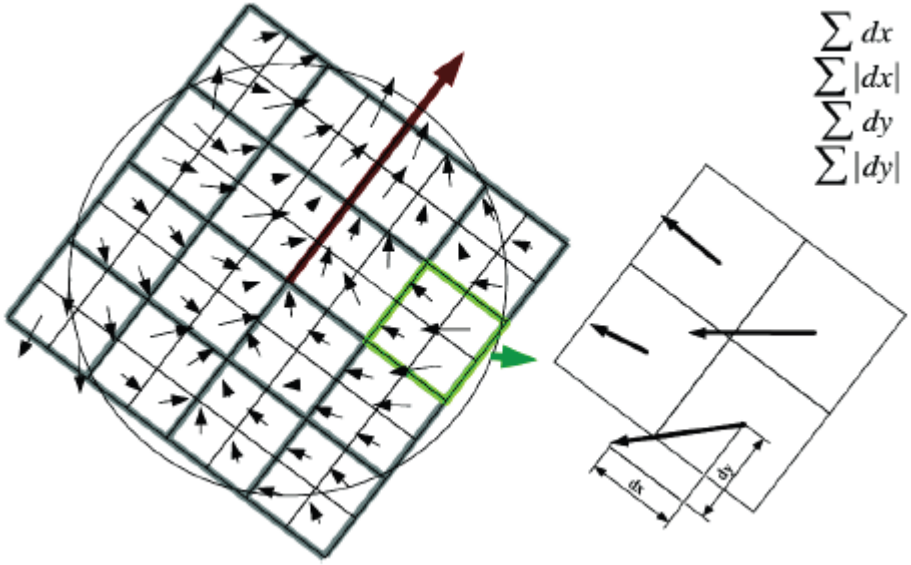
- A ciascun keypoint viene assegnata un'orientazione (per invarianza a rotazioni) nel modo seguente:
  - Si calcola la risposta (pesata) a due filtri (Haar-like) in direzione *x* e *y*. La risposta viene calcolata in un intorno circolare del punto di dimensione 6*s* dove *s* è la scala caratteristica del punto.
  - La risposta ai due filtri per ciascun punto dell'intorno viene rappresentata come un punto in uno spazio bidimensionale: la risposta in *x* rappresenta l'ascissa, la risposta in *y* l'ordinata. L'orientazione dominante è stimata calcolando la somma di tutte le risposte su una serie di «finestre» di dimensione $\frac{\pi}{3}$.





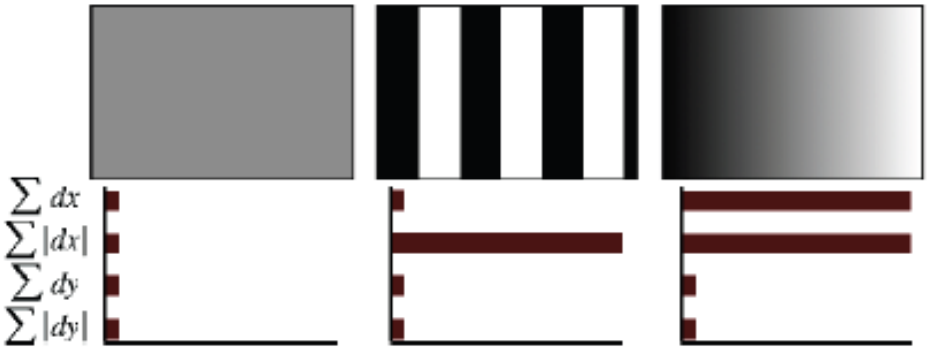
SURF: descrittore

- Il primo passo per la costruzione del descrittore consiste nella definizione di una regione quadrata centrata nel punto di interesse e orientata come indicato dall'orientazione assegnata al punto. La dimensione della finestra è 20*s*.
- La regione viene poi suddivisa in 4 × 4 sottoregioni e per ciascuna regione si calcola la risposta ai filtri Haar-like in corrispondenza di una griglia di punti; la risposta nella direzione orizzontale (ruotata) è definita d_x, mentre quella in verticale è d_y.



SURF: descrittore (1)

- Ciascuna risposta viene pesata con una Gaussiana centrata nel punto di interesse e un primo set di feature si ottiene sommando le risposte pesate d_x e d_y in ciascuna sottoregione.
- Si memorizza anche la somma del valore assoluto delle risposte, $|d_x|$ e $|d_y|$ per tenere traccia della polarità del cambiamento di intensità.
- Per ciascuna sottoregione si ottiene quindi un descrittore $\mathbf{v} = (\sum d_x, \sum d_y, \sum |d_x|, \sum |d_y|)$. La concatenazione di questi valori per tutte le sottoregioni porta a un descrittore di dimensione 64.



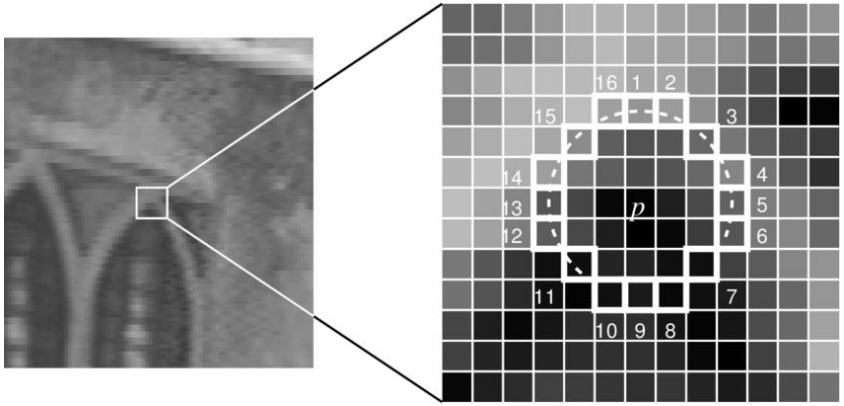
SURF: descrittore (2)

- Il descrittore ottenuto viene infine normalizzato (vettore unitario) per ottenere invarianza rispetto a variazioni di contrasto.

- Esiste una versione estesa del descrittore di dimensione 128: si ottiene calcolando le somme di d_x e $|d_x|$ separatamente per $d_y < 0$ e $d_y > 0$; lo stesso viene fatto per d_y e $|d_y|$ ottenendo così un descrittore di lunghezza doppia.

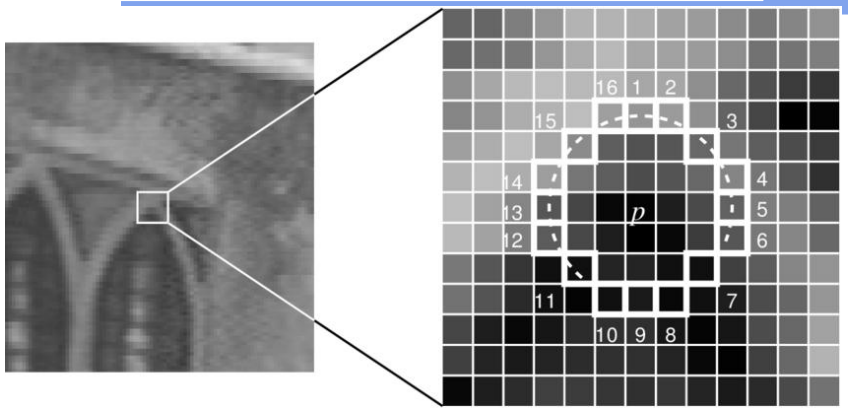
FAST keypoints

- Il keypoint FAST è un corner, identificato sulla base delle variazioni locali di luminosità.
- L'idea alla base dell'algoritmo di detection è che un punto rappresenta un corner se ha un colore molto diverso (più chiaro o più scuro) da quello dei pixel circostanti.



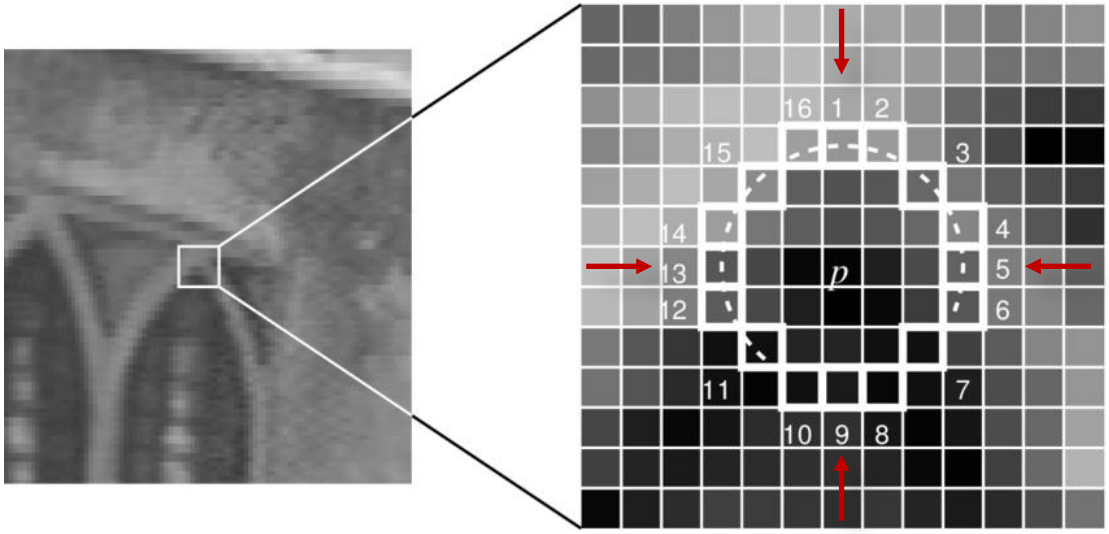
- L'approccio FAST è molto efficiente, proprio perché la procedura di detection è più semplice rispetto ad altri approcci in quanto richiede solo un confronto in termini di luminosità.

FAST: algoritmo



1. Consideriamo un pixel p nell'immagine e indichiamo con I_p la sua luminosità (gray level);
2. Fissiamo una soglia T (es. $T = 0.2 \times I_p$);
3. Fissiamo il pixel p come centro e selezioniamo i 16 pixel in un intorno circolare di raggio 3.
4. Il pixel p sarà considerato un corner se ci sono nell'intorno *N* punti consecutivi la cui luminosità è maggiore di $I_p + T$ oppure inferiore a $I_p - T$.
   N solitamente assume valori prefissati (12, 11 o 9 → FAST-12, FAST-11, FAST-9).
5. Il procedimento descritto viene iterato per tutti i pixel.

FAST: algoritmo



- È possibile velocizzare la ricerca controllando il valore dei pixel 1, 5, 9 e 13. Il pixel p è un candidato corner solo se almeno 3 di questi 4 punti soddisfano tutti una delle due condizioni ($> I_p + T$ oppure $< I_p - T$).
- I corner FAST sono spesso clusterizzati; dopo la detection è solitamente necessario adottare una procedura di non-maxima suppression.

FAST: proprietà e limitazioni

- Elevata efficienza
- Bassa ripetibilità (basato su luminosità)
- Non include informazioni direzionali, per cui non risulta invariante per rotazione
- Il raggio di analisi è fisso (3), per cui l'algoritmo non è invariante per cambiamenti di scala.

FAST: varianti (1)

- Obiettivo: raggiungere invarianza rispetto a rotazione e cambiamenti di scala.

Rotazione: assegnare un'orientazione al keypoint individuando un *centroide di intensità*:

- In una piccola patch B dell'immagine, calcolare i momenti in B:

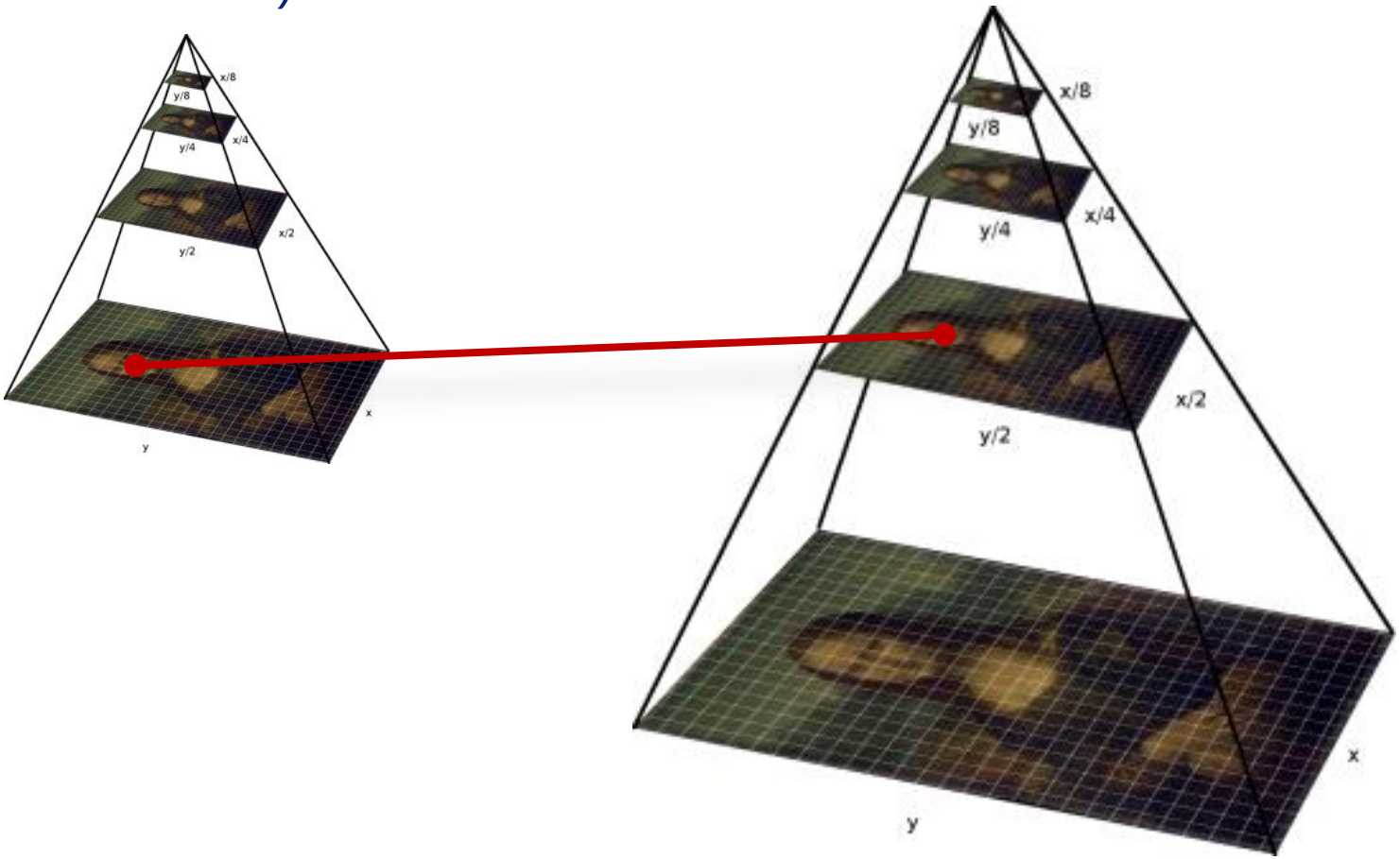
$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y),\ p,q \in \{0,1\}$$

- Calcolare il centroide della patch come $C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}}\right)$
- Connettere il centro geometrico *O* della patch con il centroide di intensità *C* ottenendo così un vettore direzione $\overrightarrow{OC}$. La direzione del keypoint è dunque data da:

$$\theta = \arctan\left(\frac{m_{01}}{m_{10}}\right)$$

FAST: varianti (2)

- ***Scala***: l'invarianza si raggiunge attraverso una ricerca dei corner all'interno di una *image pyramid* (insieme di immagini riscalate a diverse risoluzioni).



BRIEF: descrittore

- La tecnica BRIEF (**B**inary **R**obust **I**ndependent **E**lementary **F**eatures) è stata proposta in [Cal12] per la creazione di descrittori di dimensioni limitate, efficaci ed efficienti da calcolare.
- Il descrittore BRIEF è costituito da una stringa binaria creata valutando le differenze d'intensità tra coppie di pixel selezionate nell'intorno del keypoint.
- Il test τ su una patch $\mathbf{p}$ (sottoimmagine) di dimensione $S \times S$ è definito come:

$$\tau(\mathbf{p}; \mathbf{x}, \mathbf{y}) := \begin{cases} 1 \; se \; \mathbf{p}(\mathbf{x}) < \mathbf{p}(\mathbf{y}) \\ 0 \; altrimenti \end{cases}$$

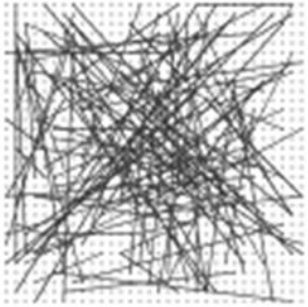
dove $\mathbf{p}(\mathbf{x})$ è il valore di intensità in una versione smoothed di $\mathbf{p}$ in posizione $\mathbf{x} = (u, v)^{\mathrm{T}}$. Si effettua una serie di test binari su n_d coppie di pixel, ottenendo così il seguente descrittore **binario** (confronto con distanza di Hamming):

$$f_{n_d}(\mathbf{p}) = \left(\tau(\mathbf{p}; \mathbf{x}_1, \mathbf{y}_1), \ldots, \tau\left(\mathbf{p}; \mathbf{x}_{n_d}, \mathbf{y}_{n_d}\right)\right)$$

BRIEF: note

- I valori di n_d suggeriti dagli autori sono 128, 256 e 512.
- Si noti che il test τ considera soltanto le informazioni relative ai pixel in esame e risulta pertanto molto sensibile al rumore. Per questo è preferibile applicare una fase di pre-processing che realizzi uno smoothing dell'immagine al fine di aumentare la ripetibilità e la stabilità dei descrittori.
- Per mantenere l'efficienza dell'approccio, per lo smoothing si utilizza un box filter da applicare al punto di interesse invece del tradizionale filtro Gaussiano.

BRIEF: coppie di pixel (1)

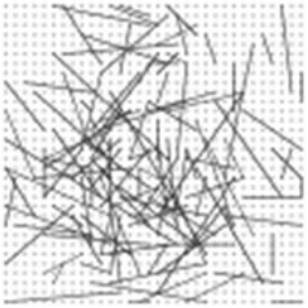


$$(\mathbf{X},\mathbf{Y}) \sim Uniform\left(-\frac{S}{2},\frac{S}{2}\right)$$



$$(\mathbf{X},\mathbf{Y}) \sim Gaussian\left(0,\frac{1}{25}S^2\right)$$

Punti campionati con distribuzione Gaussiana isotropica.

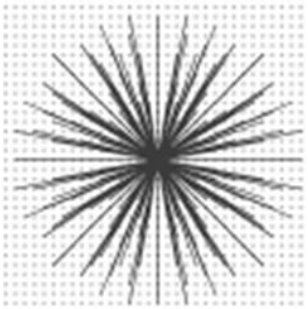


$$\mathbf{X} \sim Gaussian\left(0,\frac{1}{25}S^2\right)\ \mathbf{Y} \sim Gaussian\left(\mathbf{x}_i,\frac{1}{100}S^2\right)$$

Il campionamento avviene in due fasi: prima si campiona la posizione $\mathbf{x}_i$ con distribuzione Gaussiana centrata sul punto, poi si campiona la seconda coordinata con distribuzione Gaussiana centrata in $\mathbf{x}_i$.

BRIEF: coppie di pixel (2)

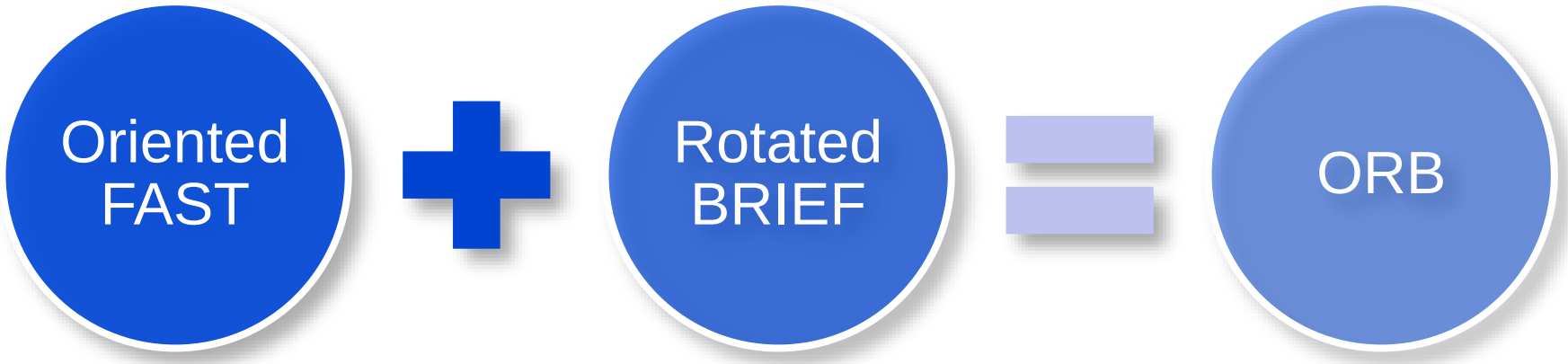




I punti $(\mathbf{x}_i, \mathbf{y}_i)$ sono campionati in modo casuale in corrispondenza di posizioni discrete di una griglia polare approssimata.

$\forall i: \mathbf{x}_i = (0,0)^T$ e $\mathbf{y}_i$ assume tutti i possibili valori su una griglia polare approssimata contenente n_d punti.

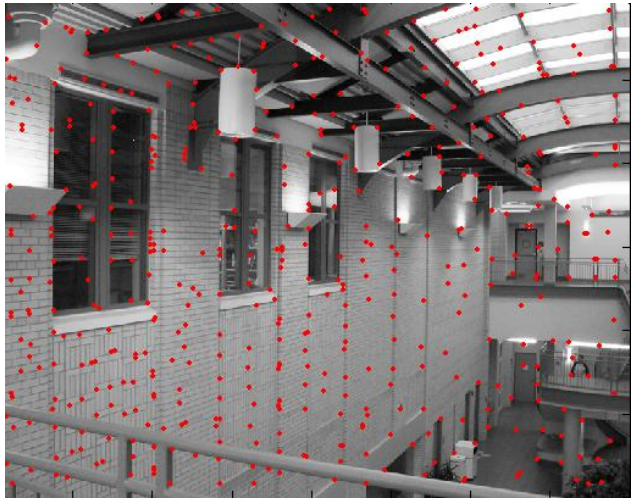
ORB: keypoint e descrittore

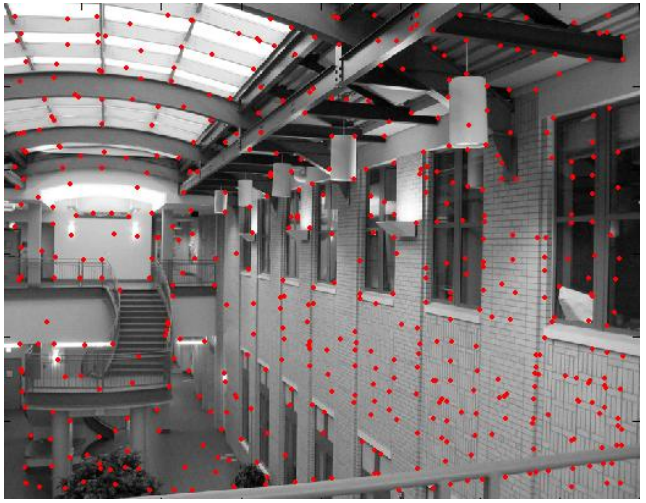


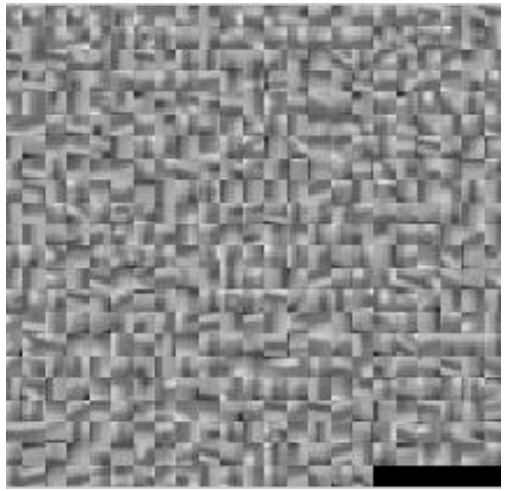
Keypoint FAST con associate informazioni di scala e orientazione

Descrittori BRIEF calcolati orientando le patch sulla base dell'orientazione dei keypoint

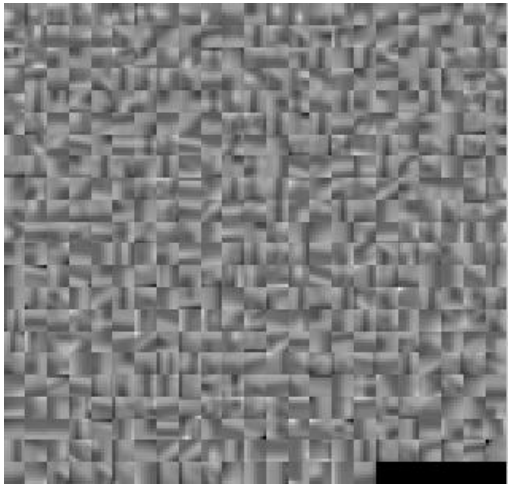
Feature matching







?



Feature matching

1. Rilevare i keypoint e calcolarne i descrittori
   SIFT, SURF, BRIEF, ecc…

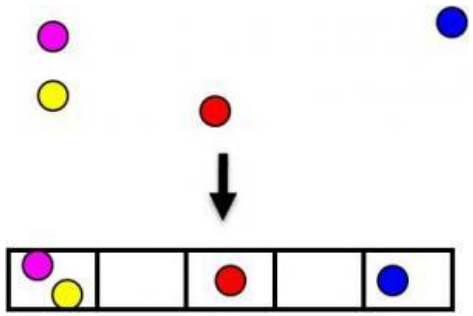
2. Determinare un set di presunti match
   Ricerca esaustiva o utilizzo di strutture dati

3. Sulla base dei migliori match determinare la trasformazione ottimale.
   Ransac

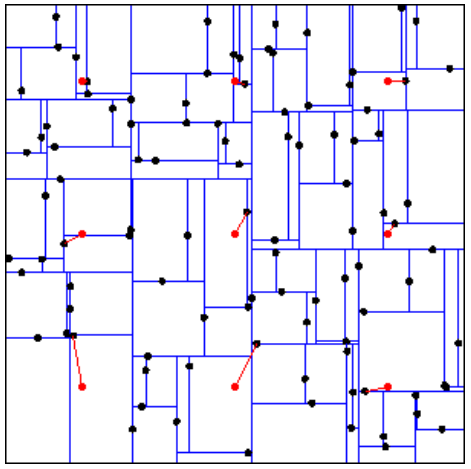
Feature matching: possibili approcci

- Come confrontare due set di punti con descrittori locali?
  - Ricerca esaustiva: confronto di ogni feature in un'immagine con *tutte* le feature dell'altra;
  - Per limitare la complessità computazionale è necessario restringere il numero di confronti da effettuare:
    - Hashing: applicazione di una funzione hash per organizzare i descrittori in *bucket* contenenti oggetti «simili»;
    - Index-based Nearest neighbor search: es. KD-Tree
  - Problema: presenza di outliers.

Hashing



KD-Tree



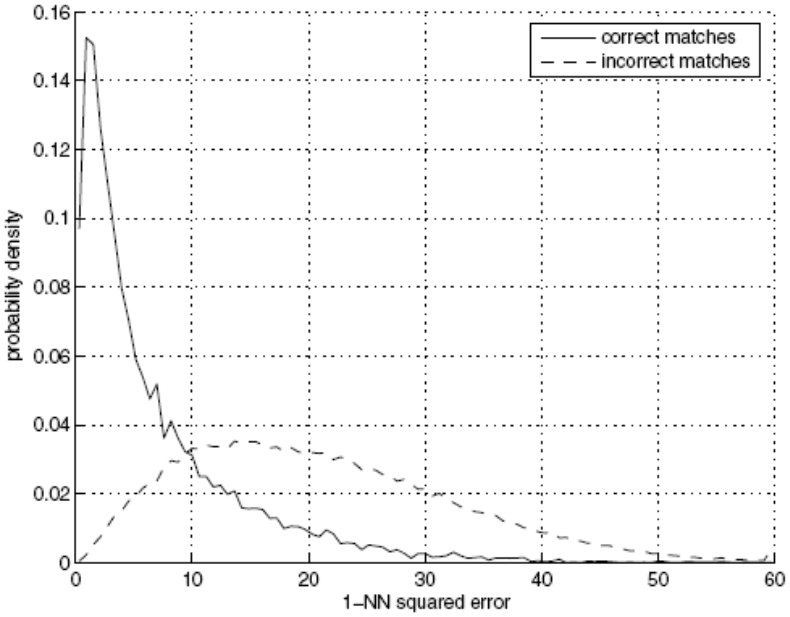
Come selezionare i match presunti?

- Per ciascun keypoint, il suo *nearest neighbor* è il keypoint che ha distanza euclidea minima;
- Molte feature non avranno un match corretto (background, rumore, occlusioni, ecc…)…

- Soglia globale sulla distanza tra descrittori?

$$SSD(patch_1, patch_2) < thr$$

Come fissare la soglia?



Come selezionare i match presunti?

- L'adozione di una soglia globale non dà buoni risultati;
- È possibile scartare alcuni match errati osservando la distanza dal nearest neighbor (1-NN) in rapporto alla distanza dal secondo keypoint più simile (2-NN).

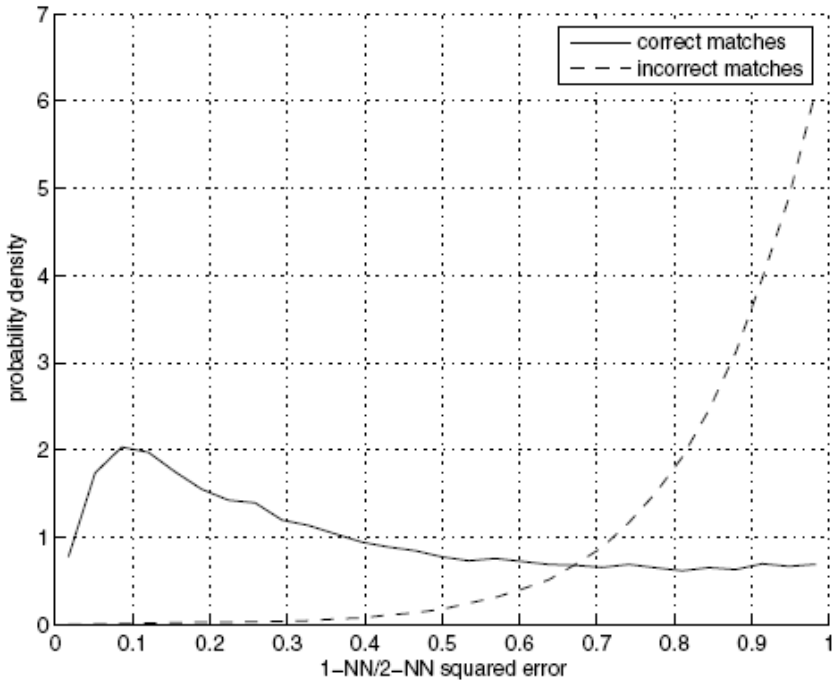
Lowe's ratio test

$$\frac{1NN}{2NN} < t$$

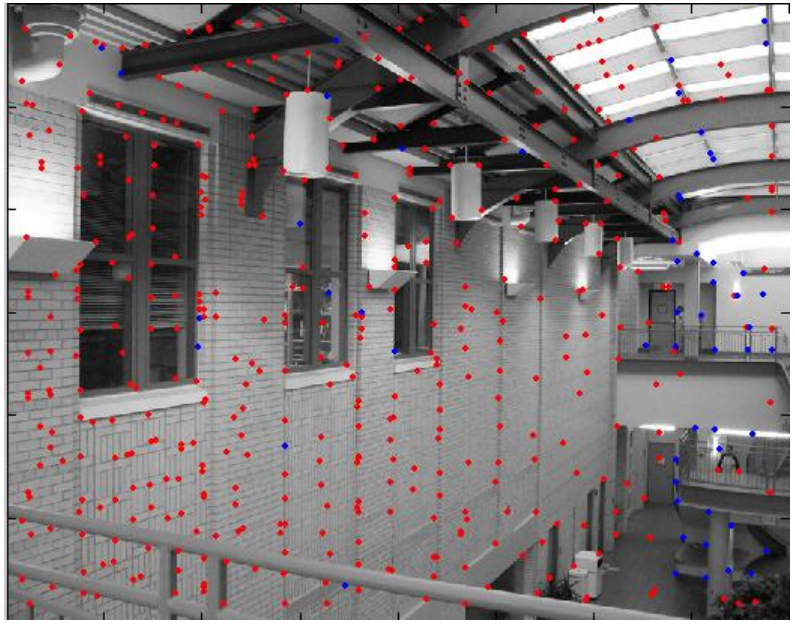
$t = 0.8$ nel paper

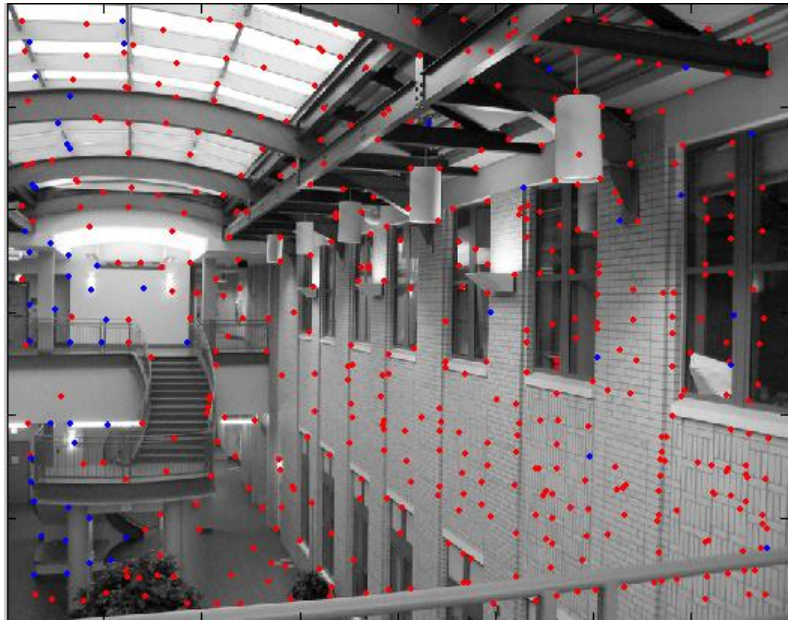
Rapporto **basso**: il primo candidato è molto più vicino del secondo.

Rapporto **vicino a 1**: i due candidati sono quasi equivalenti.



La trasformazione ottimale



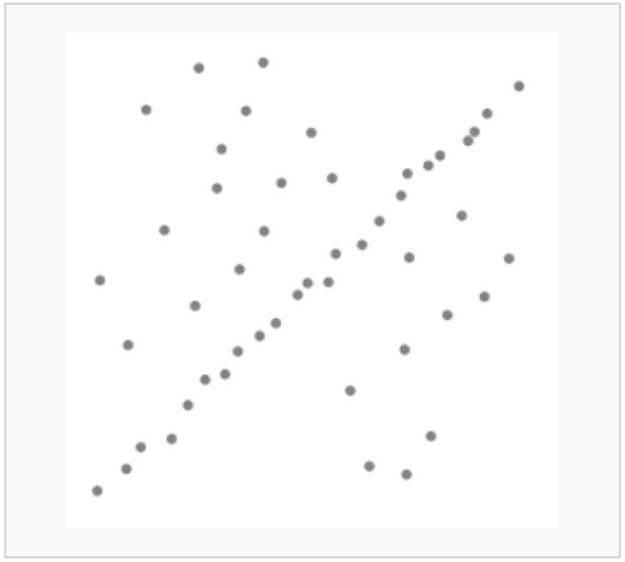


- Come possiamo calcolare la trasformazione ottimale a partire dai match presunti?
  - Problema: presenza di outlier

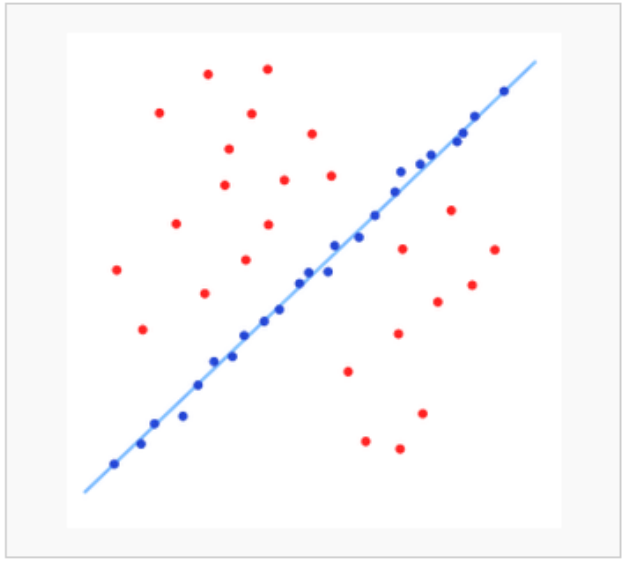
Ransac: algoritmo generale

L'algoritmo RAndom SAmple Consensus (RANSAC) è un metodo iterativo che permette di stimare un modello (es. parametri di trasformazione) a partire da un insieme di osservazioni in presenza di outliers.

L'idea alla base del metodo è che i dati si dividono in *inliers*, ovvero dati la cui distribuzione è compatibile con i parametri del modello, e *outliers* che non sono rappresentati correttamente dal modello.



Insieme di punti (con outlier)



Rette individuata con RANSAC

In generale, dati *N* punti x_i (osservazioni), assumendo che essi siano generati da un modello con parametri Θ, l'algoritmo cerca di stimare Θ.

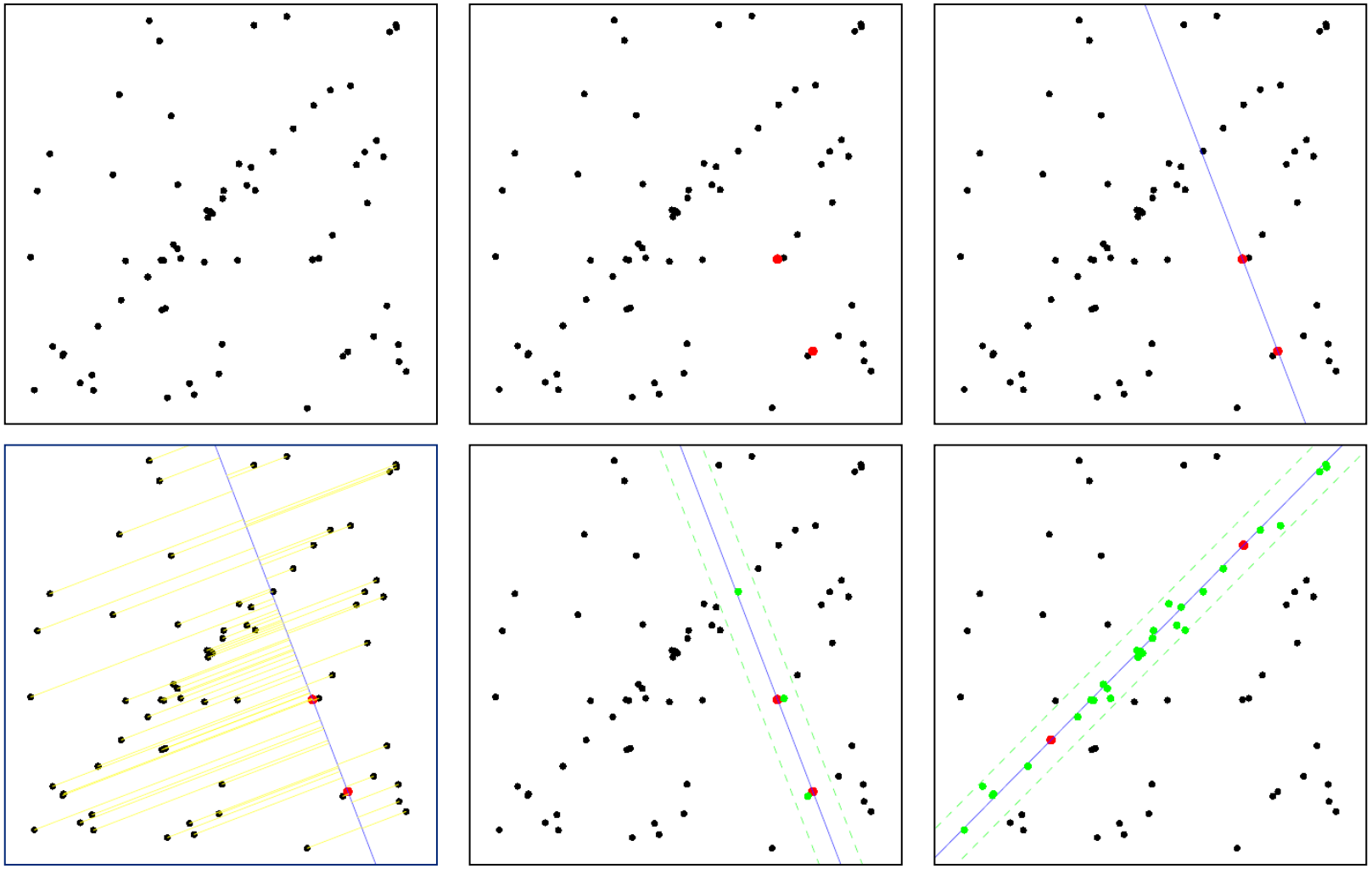
Ransac: algoritmo generale

Ripeti *K* volte:

1. Scegli *n* campioni casuali;
2. Adatta i parametri Θ agli *n* campioni;
3. Per ciascuno dei restanti *N-n* punti, calcola la distanza dal modello e conta in numero *c* di inliers

L'output è il modello Θ con *c* più elevato.

Ransac: esempio



Ransac: algoritmo generale

Ripeti K volte:

Quante volte?

1. Scegli *n* campioni casuali;

Quanti?

2. Adatta i parametri Θ agli *n* campioni;

3. Per ciascuno dei restanti *N-n* punti, calcola la distanza dal modello e conta in numero *c* di inliers

Come definirli?

L'output è il modello Θ con *c* più elevato.

Ransac: modello generico

- Ogni tipo di modello ha un ***minimal set***, ovvero un numero minimo di esempi che consentono di calcolare il modello stesso (es. linea → 2 punti).
- Le possibili trasformazioni di un'immagine sono modelli. In questo caso il minimal set è costituito da *s* coppie punti (matches).
- Per l'applicazione relativa all'allineamento di due immagini a partire da un insieme di keypoint l'obiettivo è quello di stimare la migliore matrice di trasformazione (omografia):

$$H = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}$$

Ransac e keypoints: parametri

- Numero iniziale di punti nel minimal set *s*
  - *Tipicamente il numero di esempi necessari per calcolare il modello (4 coppie di punti per omografia).*

- La soglia di distanza *t* che consente di definire un punto come *inlier.*

- Il numero di iterazioni *K* necessarie per ottenere un modello affidabile (in presenza di outliers).

Ransac: numero di sample set *K*

- Vogliamo scegliere un valore di *K* (numero di sample set, di iterazioni) tale che, con probabilità *p*, almeno un sample set non contenga nessun outlier (es. *p*=0.99).
- *e*: ipotetica percentuale di outliers → $(1 - e)$ inliers.
- *s*: *numero di punti per il calcolo del modello*
- *p*: *probabilità di successo*
- $P(s\ sample\ tutti\ inliers) = (1 - e)^s$
- $P(s\ sample\ almeno\ un\ outlier) = 1 - (1 - e)^s$
- $P(K\ sample\ set\ con\ almeno\ un\ outlier) = (1 - (1 - e)^s)^K$
  Vogliamo che questo valore sia piccolo…

$$(1 - (1 - e)^s)^K < (1 - p)$$

$$K > \frac{\log(1 - p)}{\log(1 - (1 - e)^s)}$$

N.B. il numero *K* non dipende dal numero totale di punti

Ransac: numero di samples *K*

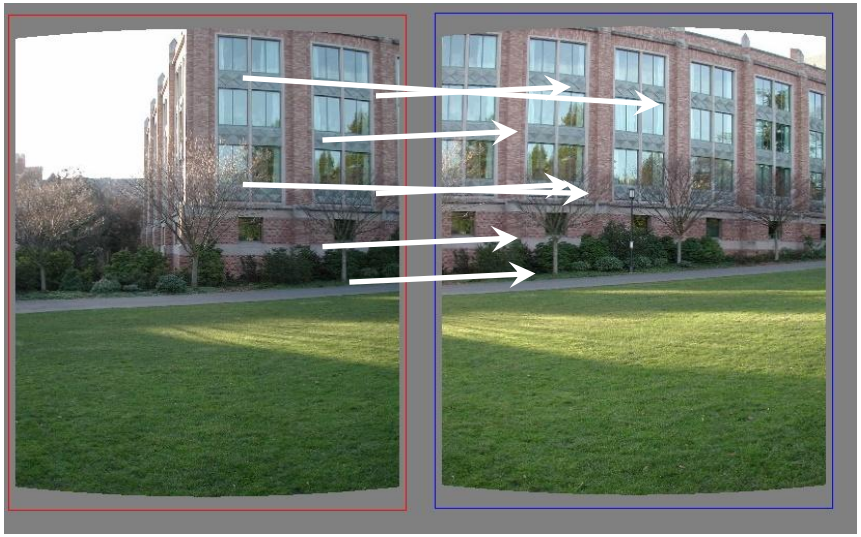
$$K > \frac{\log(1-p)}{\log(1-(1-e)^s)}$$

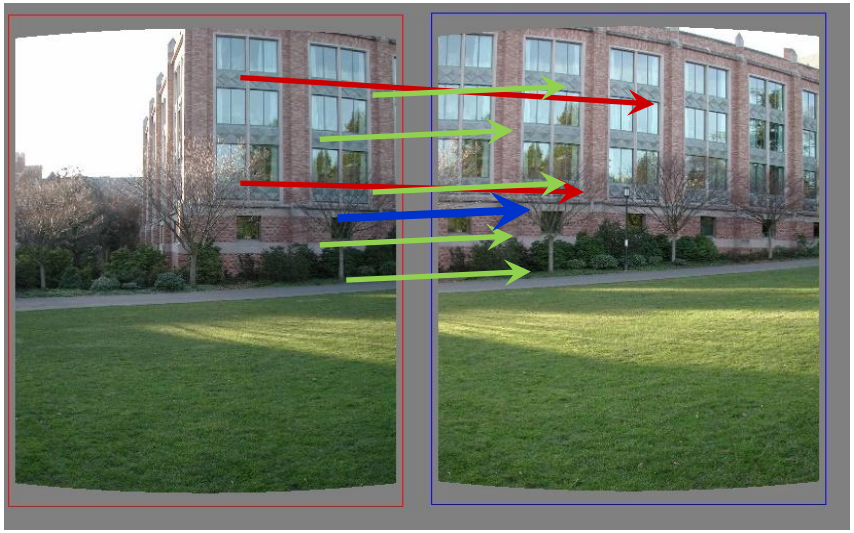
s	*e*									
	0.05	***0.10***	***0.15***	***0.20***	***0.25***	***0.30***	***0.35***	***0.40***	***0.45***	***0.50***
2	2	3	4	5	6	7	9	11	13	17
3	3	4	5	7	9	11	15	19	26	35
4	3	5	7	9	13	17	24	34	48	72
5	4	6	8	12	17	26	38	57	90	146
6	4	7	10	16	24	37	59	97	165	293
7	4	8	12	20	33	54	92	163	301	588
8	5	9	15	26	44	78	143	272	548	1177





Ransac e keypoints





Ransac e keypoints (2)

- L'obiettivo è quello di stimare la migliore matrice di trasformazione (traslazione, rotazione e scala):

$$H = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix}$$

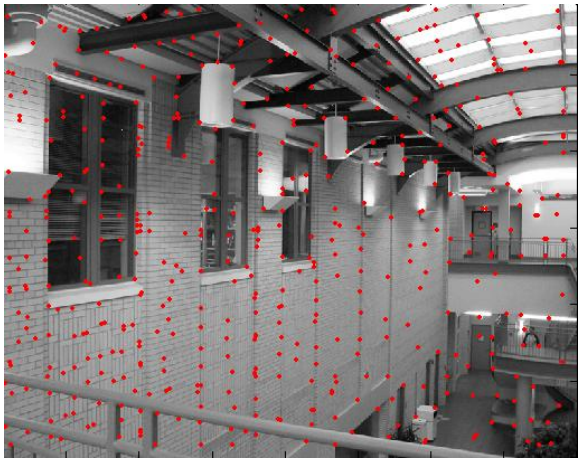
Ripeti *N* volte:

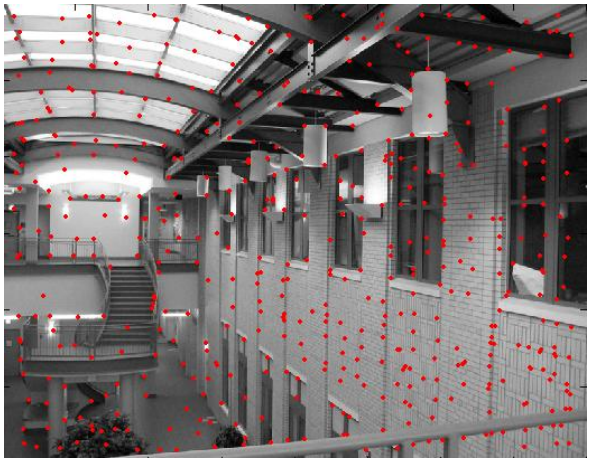
1. Seleziona 4 coppie di punti;
2. Calcola a partire da questi la matrice *H*;
3. Calcola il numero di *inlier* per i quali

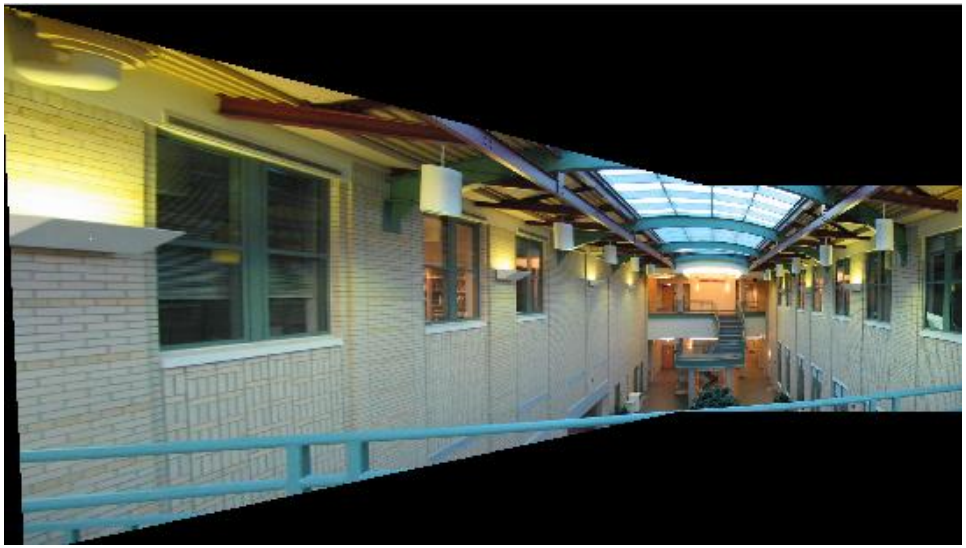
$$SSD\left(p_i', H(p_i)\right) < \epsilon$$

dove p_i' è il presunto match di p_i.

4. Seleziona la trasformazione *H* con il massimo numero di inlier (eventualmente ricalcola a partire da tutti gli inlier per una stima robusta)







Ransac: conclusioni

- Vantaggi:
  - Semplice e generale;
  - Robusto anche in presenza di un elevato numero di outlier;
  - Più facilmente applicabile a problemi con un elevato numero di parametri rispetto alla trasformata di Hough.

- Svantaggi:
  - La complessità computazionale aumenta comunque in modo significativo all'aumentare del numero di parametri del modello da stimare;
  - Non ottimale per ottenere modelli multipli.

Bibliografia

[Harris88] C.Harris e M. Stephens, "*A combined corner and edge detector"*, *Proc. Alvey Vision Conference*, pp. 147-151, 1988.

[Miko01] K. Mikolajczyk e C. Schmid, "*Indexing based on scale invariant interest points*", *proc. of the 8th International Conference on Computer Vision*, pp. 525-531, 2001.

[Lind98] T. Lindeberg, "*Feature detection with automatic scale selection*", *International Journal of Computer Vision*, vol. 30, no. 2, pp 77–116, 1998.

[Lowe99] D.G. Lowe, "*Object recognition from local scale-invariant features*", *proc. 7th International Conference on Computer Vision,* pp. 1150-1157, 1999.

[Bay08] H. Bay, A. Ess, T. Tuytelaars, L. Van Gool, "*SURF: Speeded Up Robust Features*", *Computer Vision and Image Understanding,* vol 110, n. 3, pp. 346-359, 2008.

[Cal12] M. Calonder, V. Lepetit, M. Ozuysal, T. Trzcinski, C. Strecha e P. Fua, "*BRIEF: Computing a Local Binary Descriptor Very Fast*", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 7, pp. 1281-1298, 2012.

[Sand10] K. E. A. van de Sande, T. Gevers, C. G. M. Snoek, "*Evaluating Color Descriptors for Object and Scene Recognition* ", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 32, n. 9, pp. 1582-1596, 2010.